CS 4400: Artificial Intelligence

Adversarial Search

Contents

1. Games
2. Optimal decision by Minimax
3. Speed up by alpha-beta pruning
4. Imperfect real-time decision by H-Minimax
5. Stochastic games
6. Partially observable games
7. State-of-the-arts in games

1/7. Games

- In many games someone wins, someone loses



- These types of games are competitive or adversarial
  - Zero-sum game: completely adversarial
- A game can be neither completely adversarial nor completely cooperative

Adversarial search

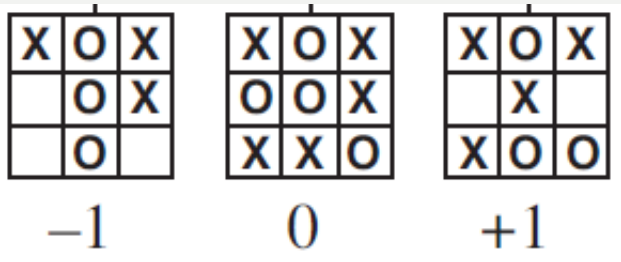
- Classical games: Chess, Checkers, Othello, Go
- Task environment:
  - Fully observable or partially observable?
  - Deterministic or stochastic? Known or unknown?
  - Sequential or episodic? Static (turn-taking) or dynamic?
  - Two-agents : adversarial or cooperative?
- Problem-solving by adversarial searching
  - Goal: find a sequence of actions that will win the game
  - Take into account the moves of the opponent, who will be working against the goal
  - Can be expanded into n-player zero-sum games

Two-player games

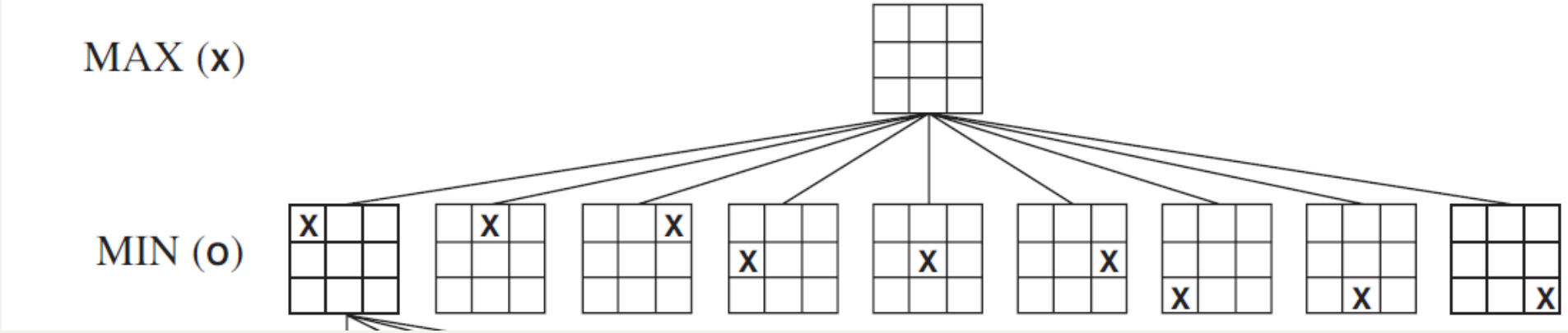
- Problem formulation
  - States. E.g., chess, checkers: a state = locations of all pieces
  - Actions(s): legal moves
  - s' = Result(s,a): transition
  - Goal-test(s) or Terminal-test(s): is it game over?
- What's new
  - Player(s): whose turn it is to move
  - Utility(s): final payoff for player 1 when the game ends in state s
    - Chess: +1 (win), 0 (draw), -1 (loss) (zero-sum)
    - Chess: +1 (win), 1/2 (draw), 0 (loss) (constant-sum)
  - Utility need not be simple win or loss: e.g., # of player 1's pieces left

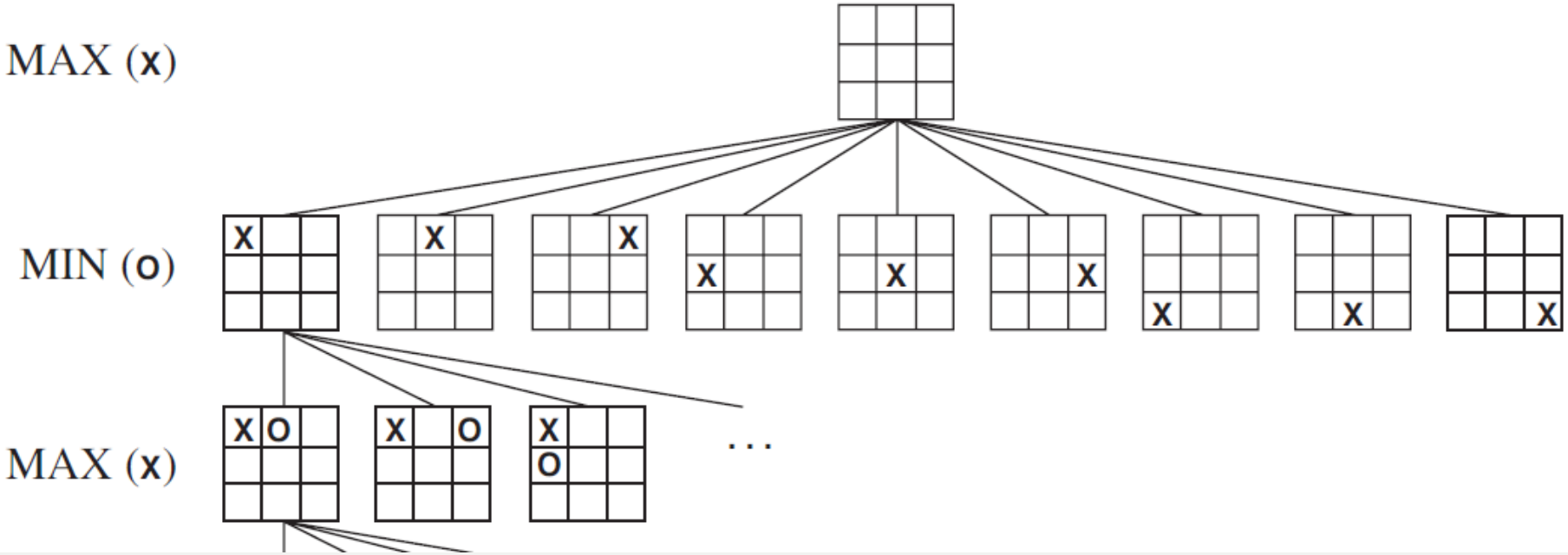
Tic-Tac-Toe

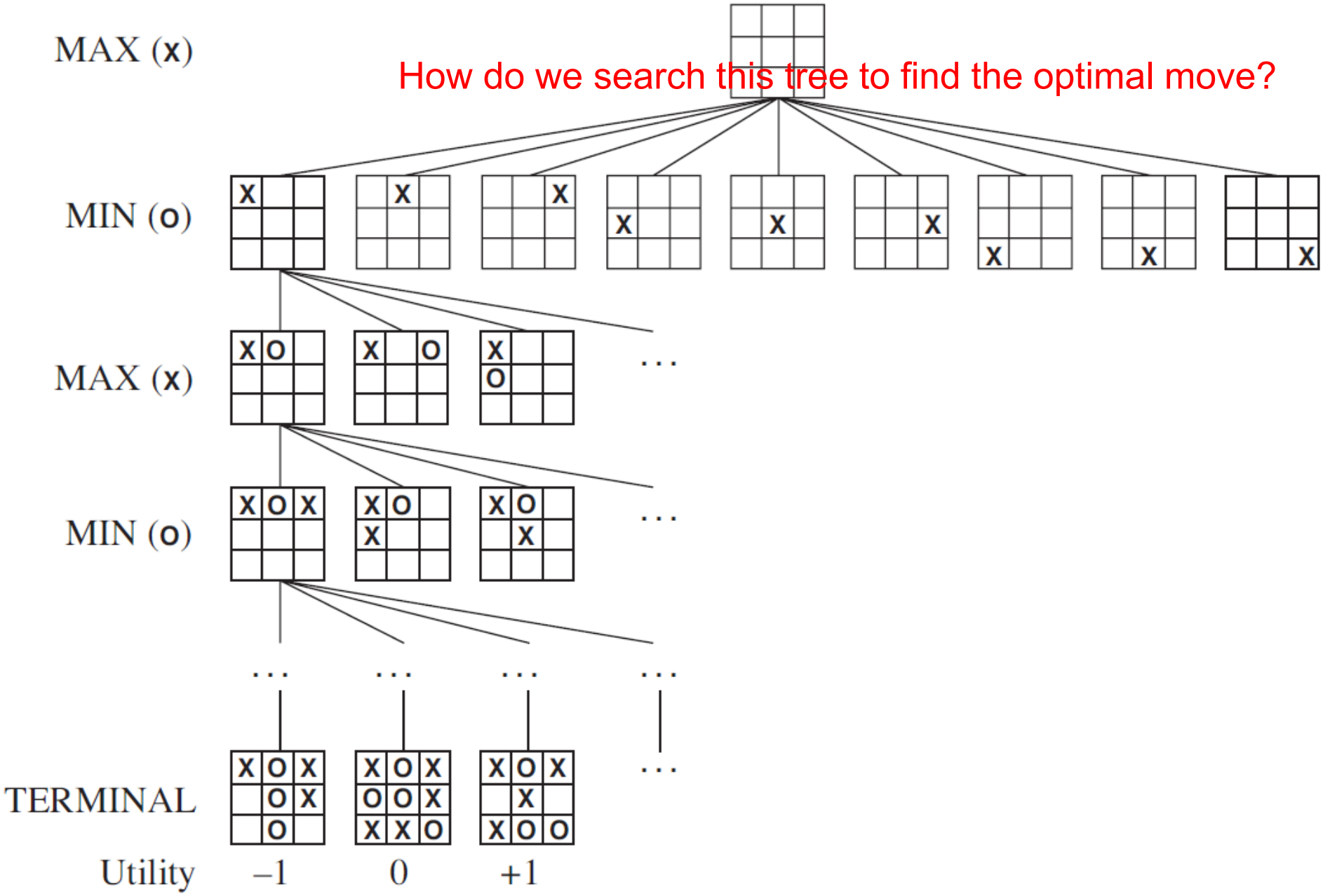
- 3 x 3 board, x (player 1) and o (player 2)
- Rules:



- Let's call the player 1 MAX, and player 2 MIN
  - player MAX will try to maximize utility
  - player MIN will try to minimize utility


MAX (x)
MIN (o)


MAX (x)
MIN (o)
MAX (x)
...



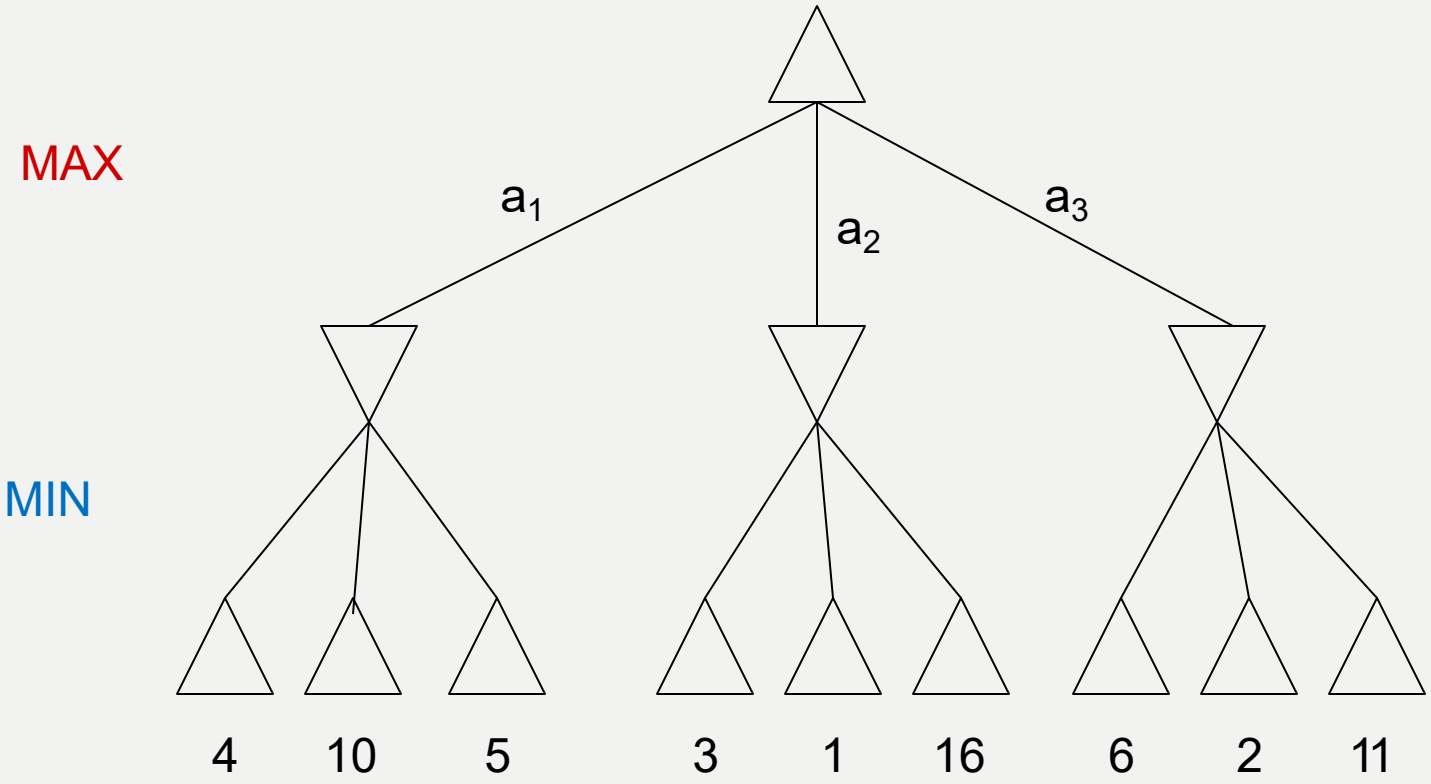
- # of states, games, terminal states
- Less number if counted up to rotation and reflection

2/7. Optimal decisions by Minimax

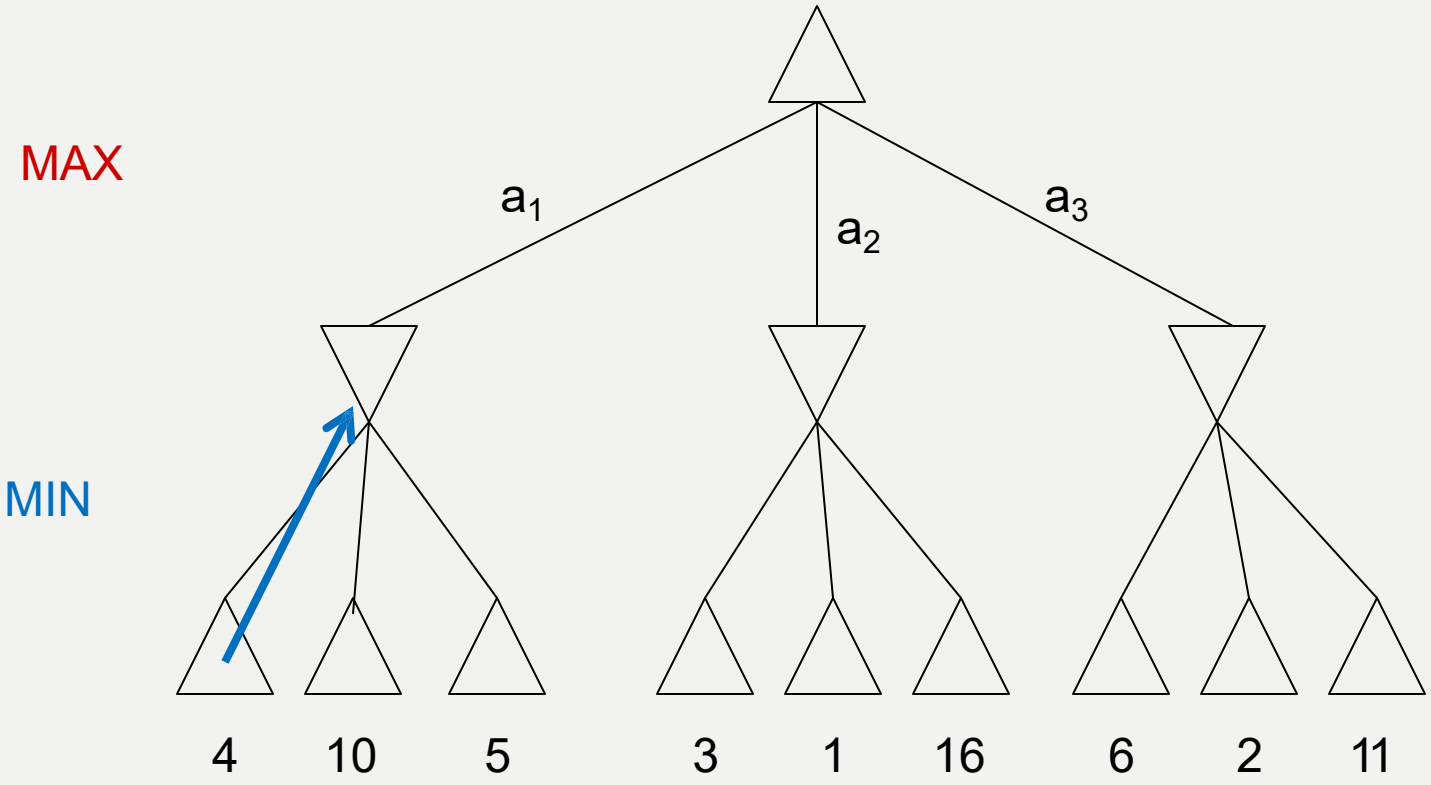
- MAX's point of view
  - At every turn, MAX will try to maximize the utility, expecting that MIN will try to minimize the utility
- Minimax(s): expected utility (for MAX) of being in state s, assuming both players play rationally
  - Cf. Utility(s) is defined only for goal states

$$\textsc{Minimax}(s) = \begin{cases} \textsc{Utility}(s) & \text{if } \textsc{Terminal-Test}(s) \\ \max_{a \in Actions(s)} \textsc{Minimax}(\textsc{Result}(s,a)) & \text{if } \textsc{Player}(s) = \textsc{max} \\ \min_{a \in Actions(s)} \textsc{Minimax}(\textsc{Result}(s,a)) & \text{if } \textsc{Player}(s) = \textsc{min} \end{cases}$$

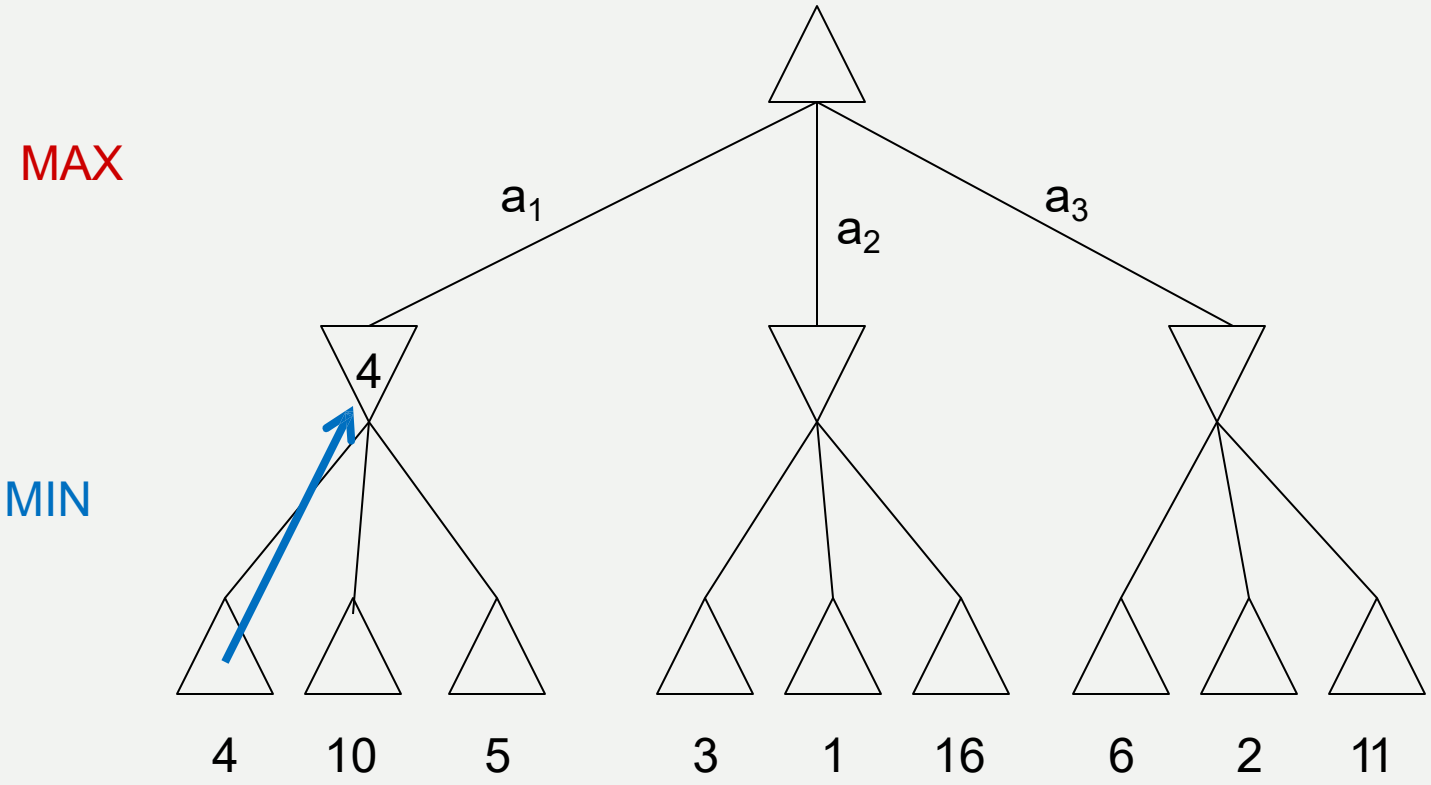
A sample game tree



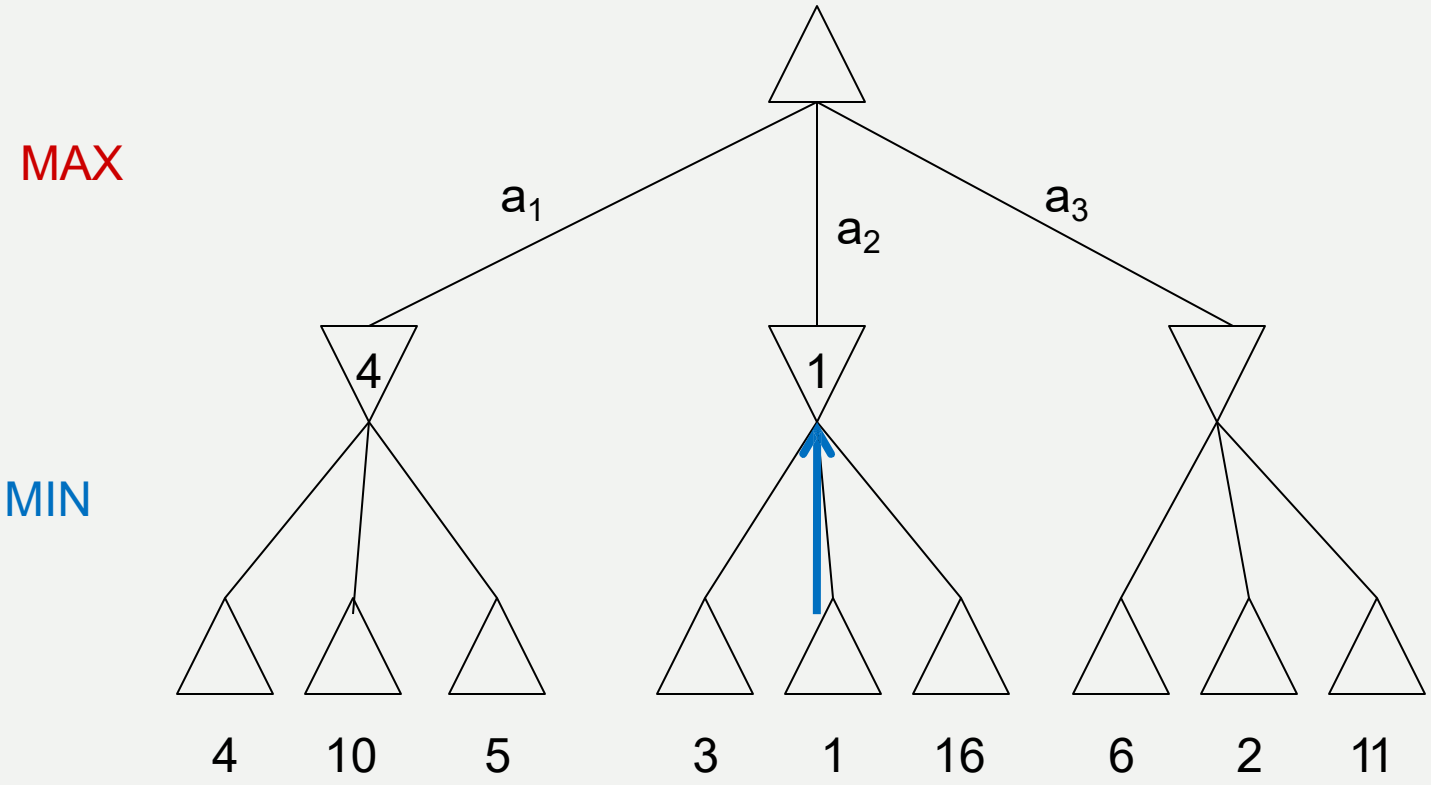
Minimax in action



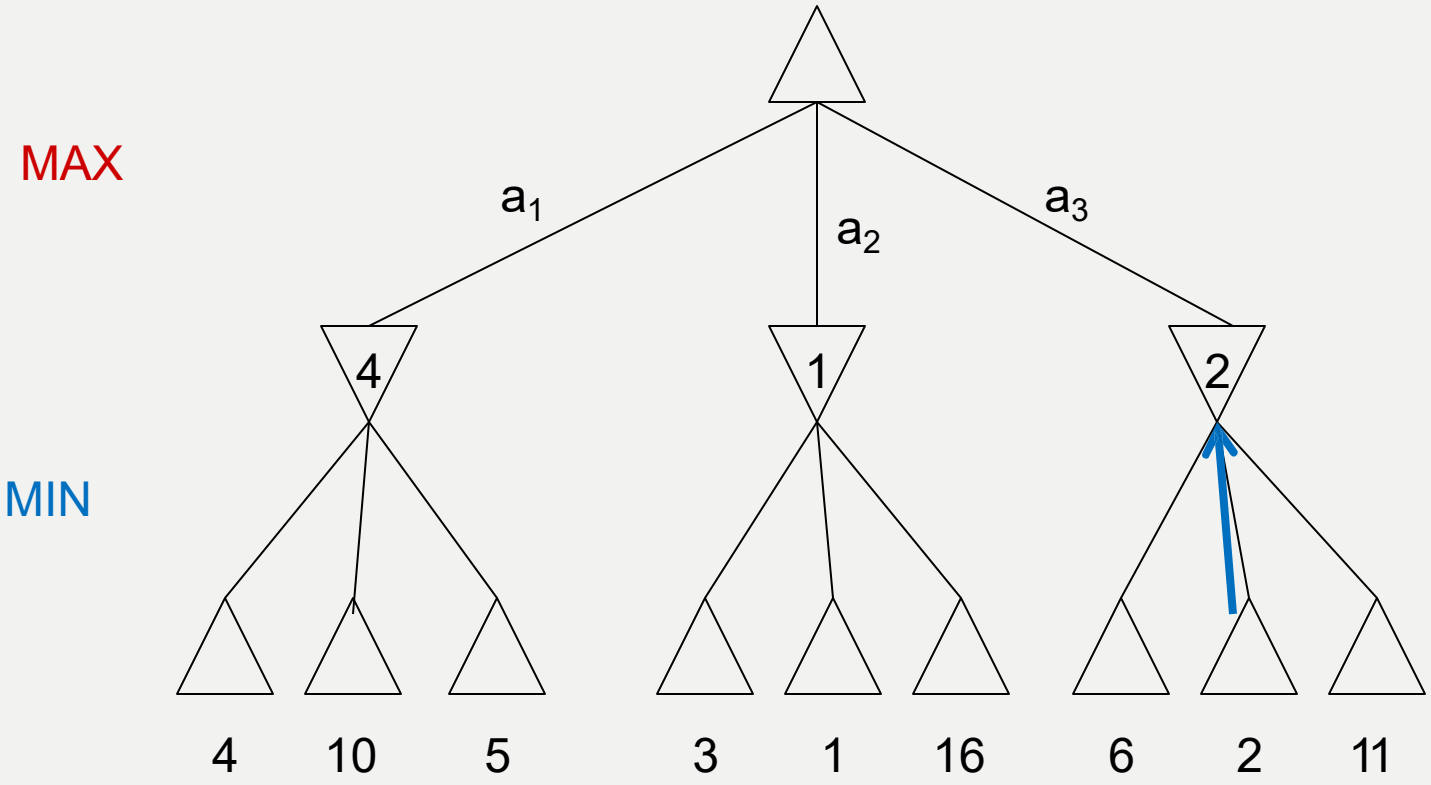
Minimax in action



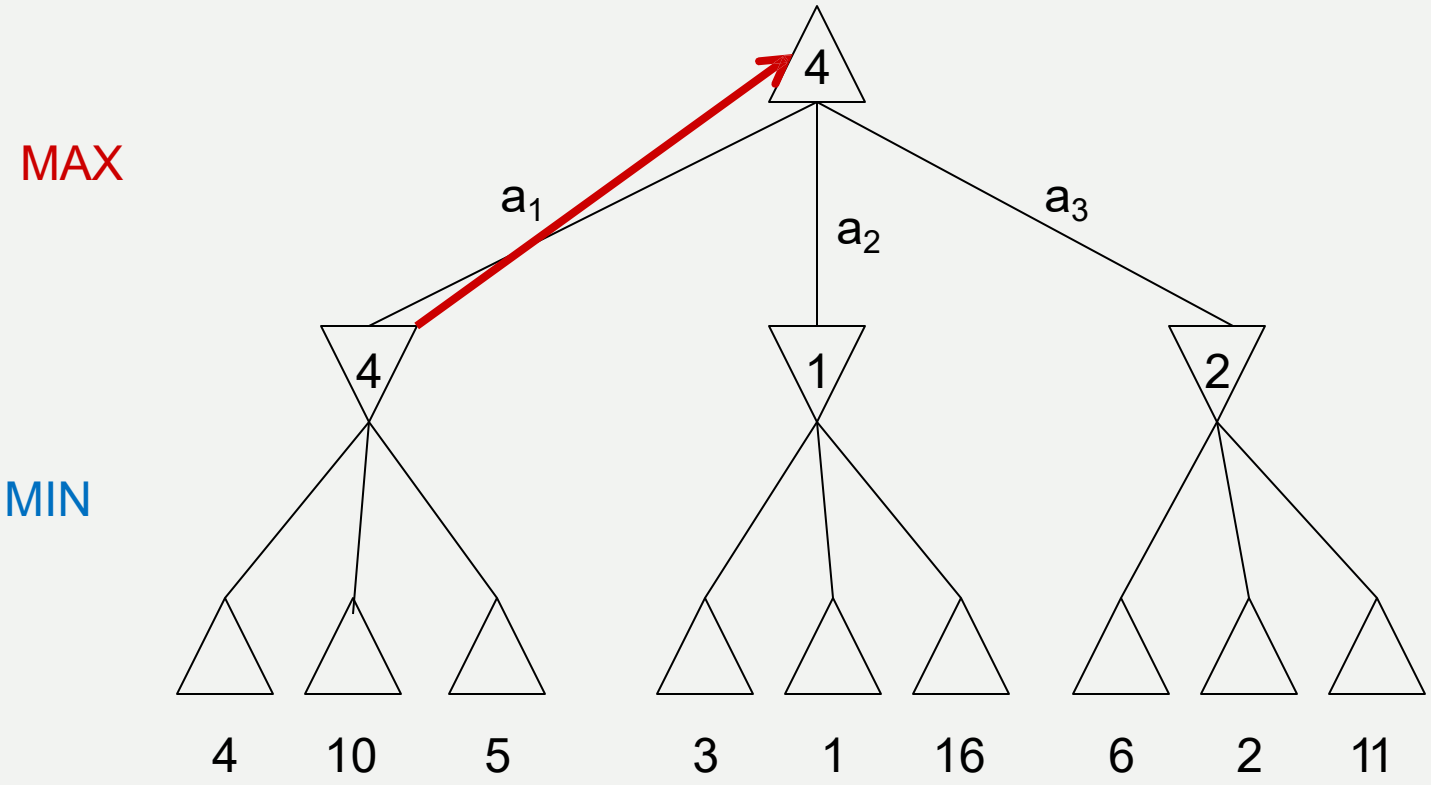
Minimax in action



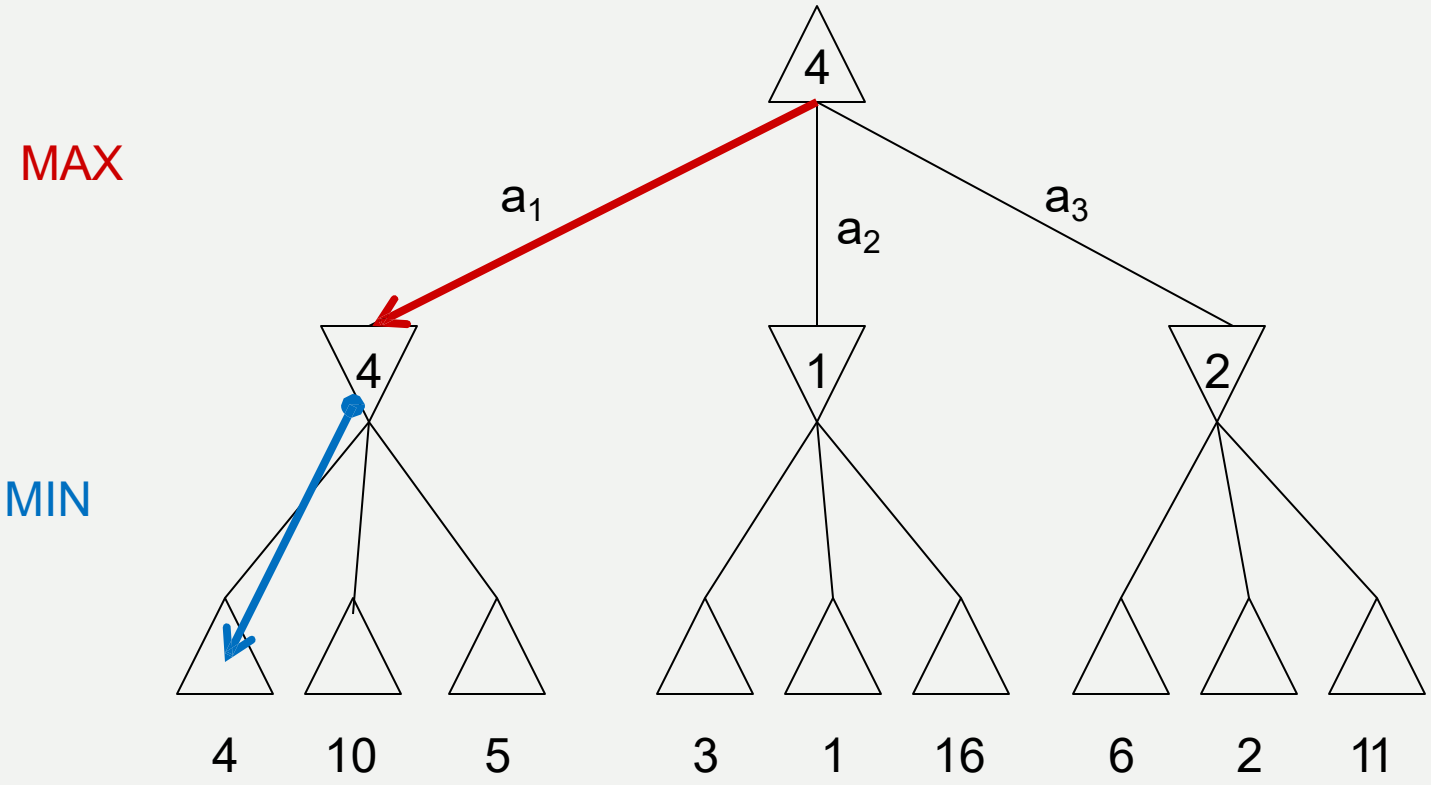
Minimax in action



Minimax in action

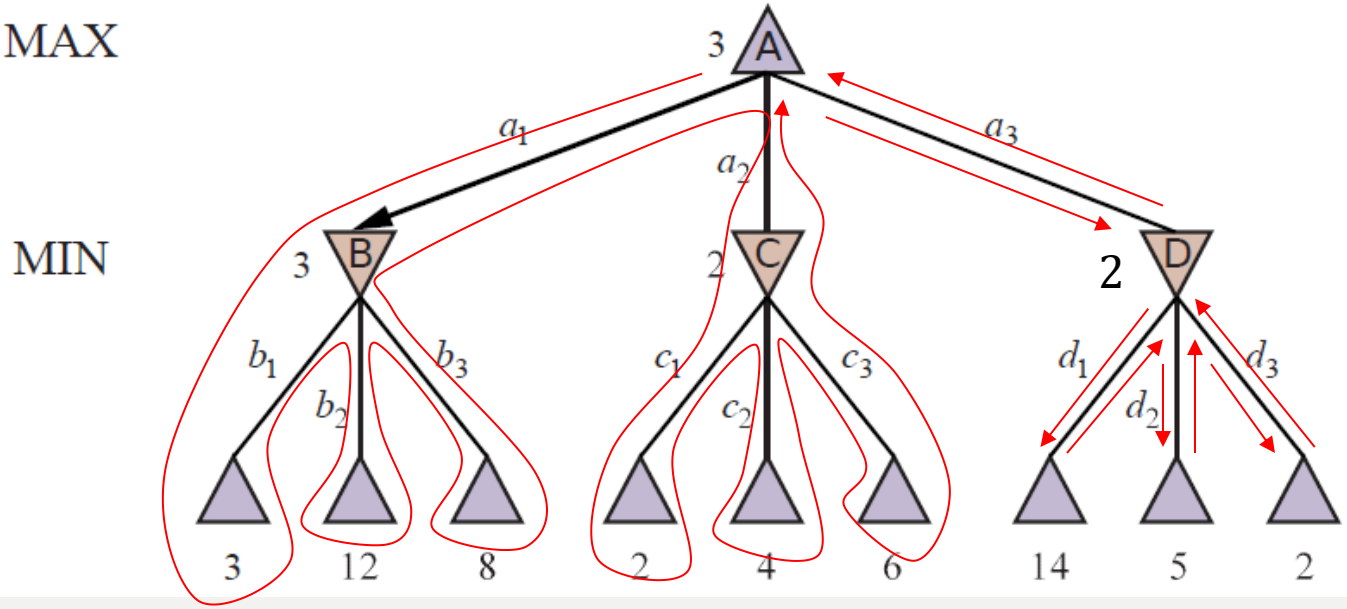


Minimax in action



Another Example

Depth-first search with backed-up value on return from a node.



Minimax algorithm

```
function MINIMAX-DECISION(state) returns an action
  return arg max_{a ∈ ACTIONS(s)} MIN-VALUE(RESULT(state, a))
```

```
function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s, a)))
  return v
```

```
function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← ∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s, a)))
  return v
```

Minimax algorithm (detailed version)

```
function MINIMAX-SEARCH(game, state) returns an action
  player ← game.TO-MOVE(state)
  if player = MAX then value, move ← MAX-VALUE( game, state )
                  else  value, move ← MIN-VALUE( game, state )
  return move

function MAX-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← −∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MIN-VALUE(game, game.RESULT(state, a))
    if v2 > v then
      v, move ← v2, a
  return v, move

function MIN-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← +∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MAX-VALUE(game, game.RESULT(state, a))
    if v2 < v then
      v, move ← v2, a
  return v, move
```

Minimax algorithm

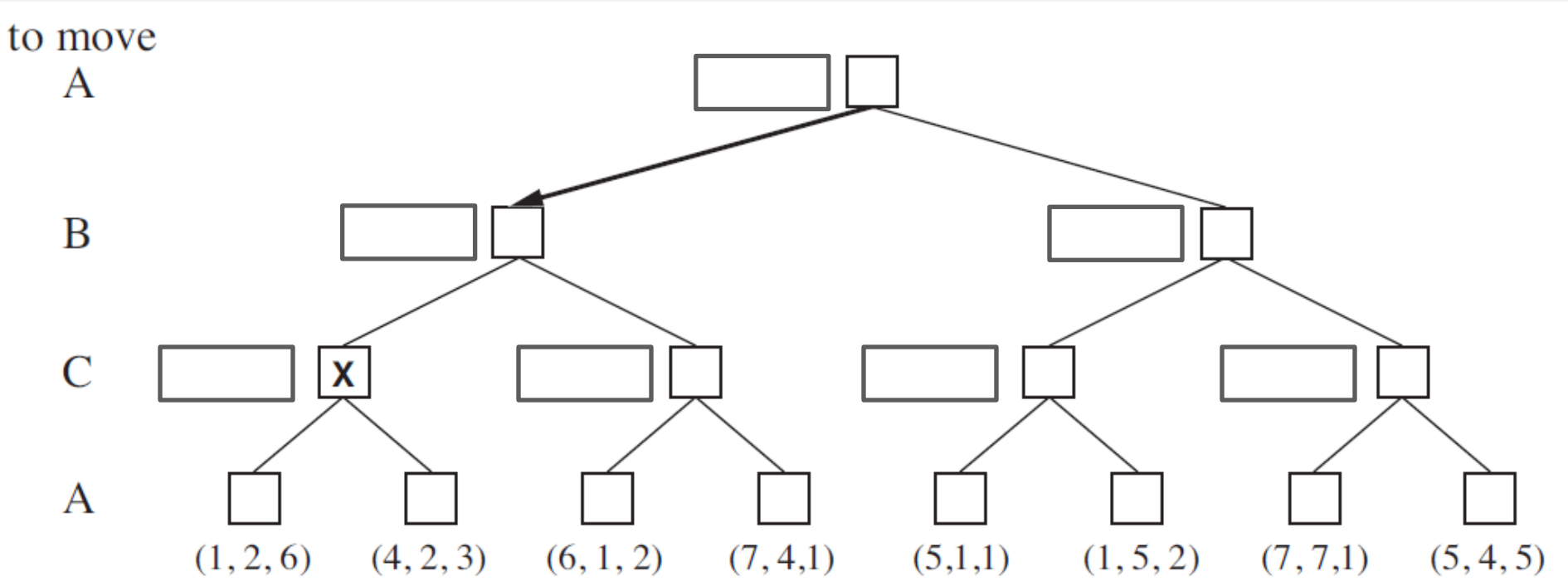
- Minimax performs depth-first search with value back-up
  - Complete? Yes, for a finite tree
  - What if the opponent is not rational?
    - It's okay - minimax is the worst-case solution.
  - Time complexity: $O(b^m)$
  - Space complexity: O(bm) *(depth-first exploration)*

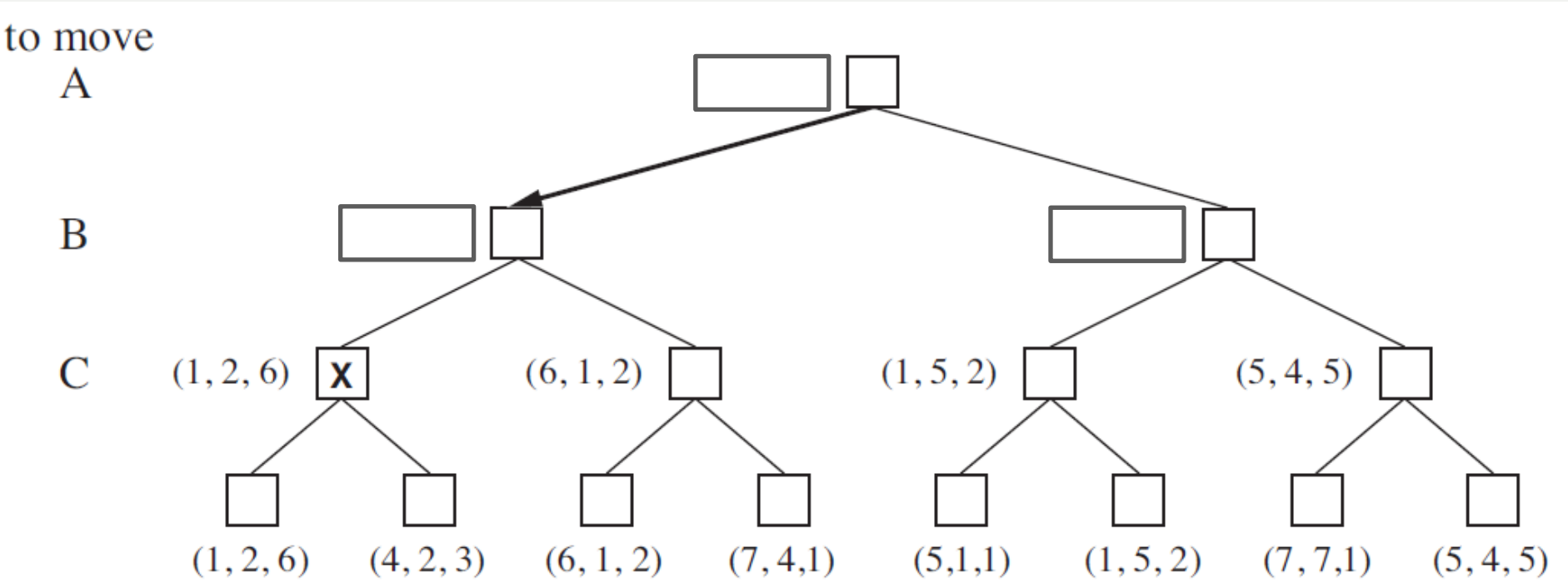
  - *For chess, b ≈ 35, m ≈100 for "reasonable" games*
    exact solution completely infeasible

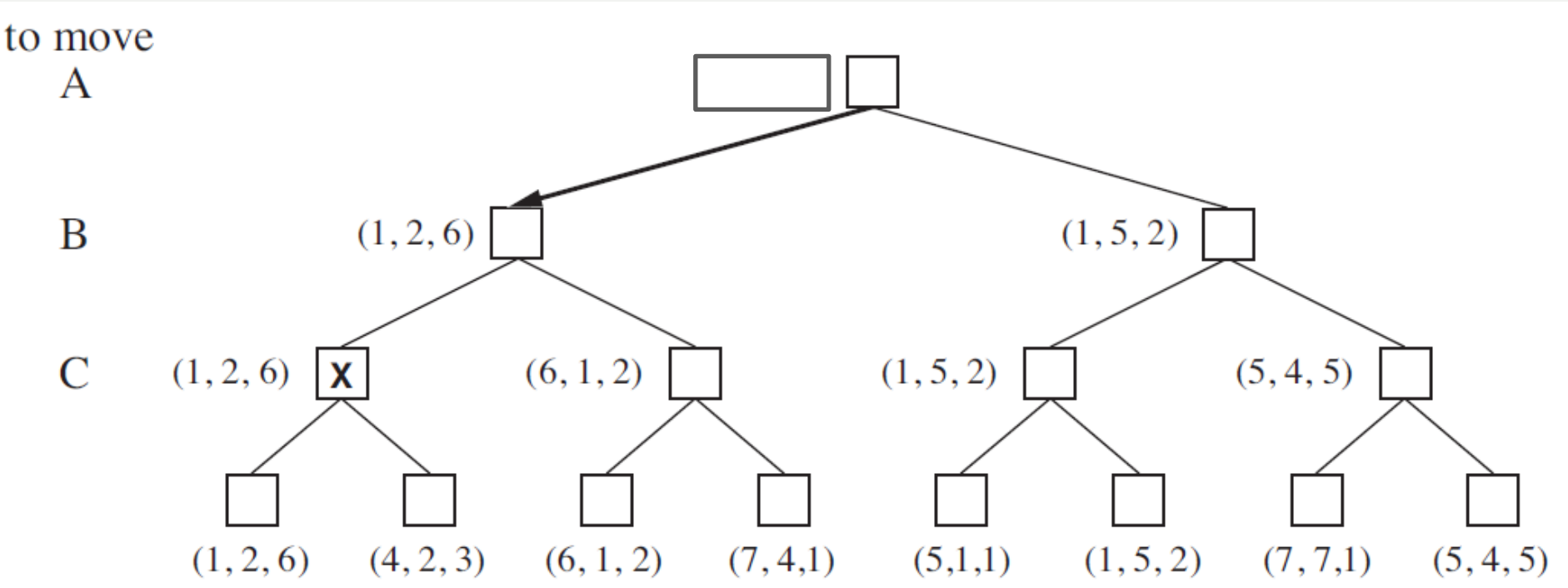
Optimal decision in multiplayer games

- Assume three players A, B, C
- Utility vector $<v_a, v_b, v_c>$(s) instead of utility value
  - v_a : final payoff for player a when game ends in state s
  - v_b : final payoff for player b …
  - v_c : final payoff for player c …

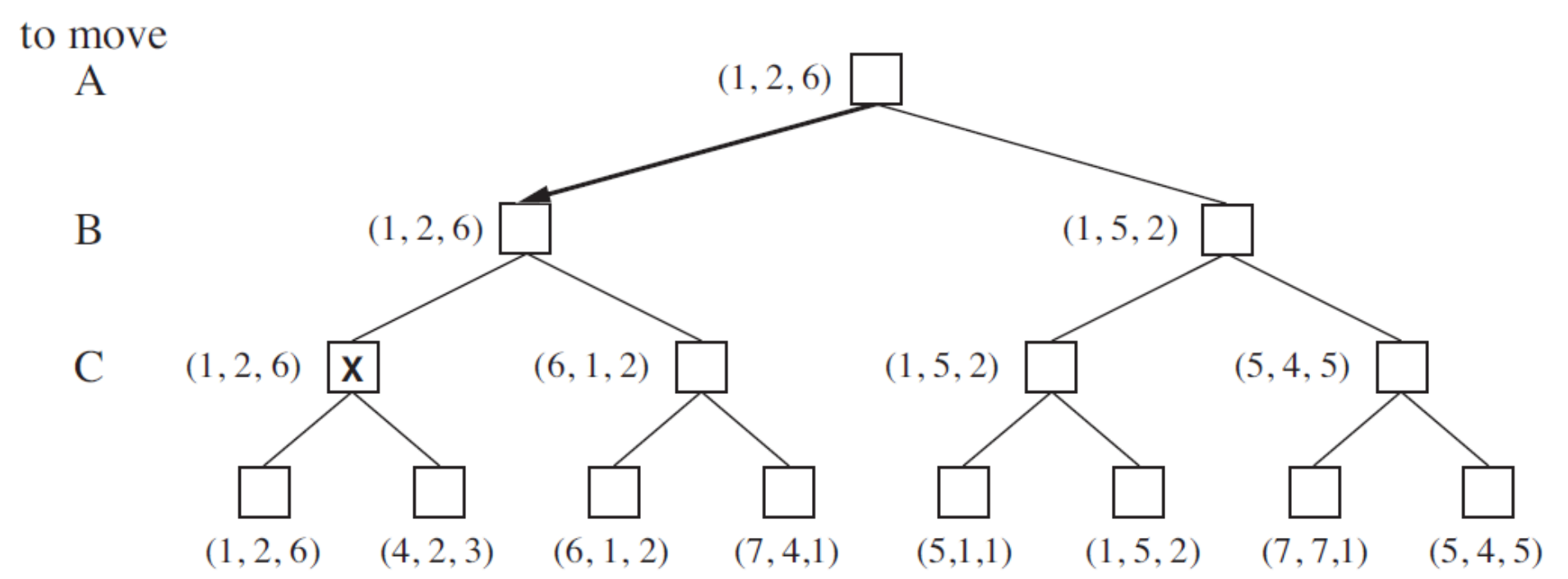
- Minimax decision for n-player game
  - Each player chooses an action that maximizes the minimax value for that player


to move
A
B
C
X
A
(1, 2, 6)
(4, 2, 3)
(6, 1, 2)
(7, 4,1)
(5,1,1)
(1, 5, 2)
(7, 7,1)
(5, 4, 5)


to move
A
B
C
(1, 2, 6)
X
(6, 1, 2)
(1, 5, 2)
(5, 4, 5)
(1, 2, 6)
(4, 2, 3)
(6, 1, 2)
(7, 4,1)
(5,1,1)
(1, 5, 2)
(7, 7,1)
(5, 4, 5)


to move
A
B
C
(1, 2, 6)
(1, 5, 2)
(1, 2, 6)
X
(6, 1, 2)
(1, 5, 2)
(5, 4, 5)
(1, 2, 6)
(4, 2, 3)
(6, 1, 2)
(7, 4,1)
(5,1,1)
(1, 5, 2)
(7, 7,1)
(5, 4, 5)

The first three plies of a game tree with three players (A, B, C).



3/7. Speed up by alpha-beta pruning

- State space is huge in real games
  - Chess: branching factor of ~35, 50 moves by each player → Time complexity ~ 10^{154} nodes
  - Exact decision-making by minimax is infeasible
- How to avoid evaluating a full tree ?
  - Stop evaluating a path when it becomes obvious that it is worse than a previous path
- Keep track of best/worst values at every node
  - [α, β]: MAX can get at least α and at most β currently at a node

Alpha and Beta Values

Alpha-beta pruning gets its name from two extra parameters α, β

α = the *highest*-value (i.e., the best choice) so far along a path for the player MAX. // MAX's best score so far.

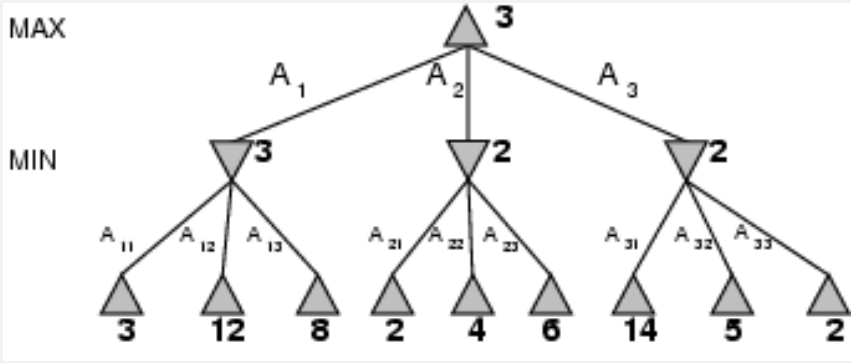
eventual value $\geq \alpha$ "at least"

β = the *lowest*-value (i.e., the best choice) so far along a path for the player MIN. // MIN's best score so far.

eventual value $\leq \beta$ "at most"

- Update the values of α and β as the search goes along.
- Prune the remaining branches
  - at a MIN node with current value $\leq \alpha$ (because its parent, a MAX node, has from another child or is passed on the value α already).
  - at a MAX node with current value $\geq \beta$ (because its parent, a MIN node, has or is passed on the value β already).

α-β pruning example



MAX

MIN

≥ 3

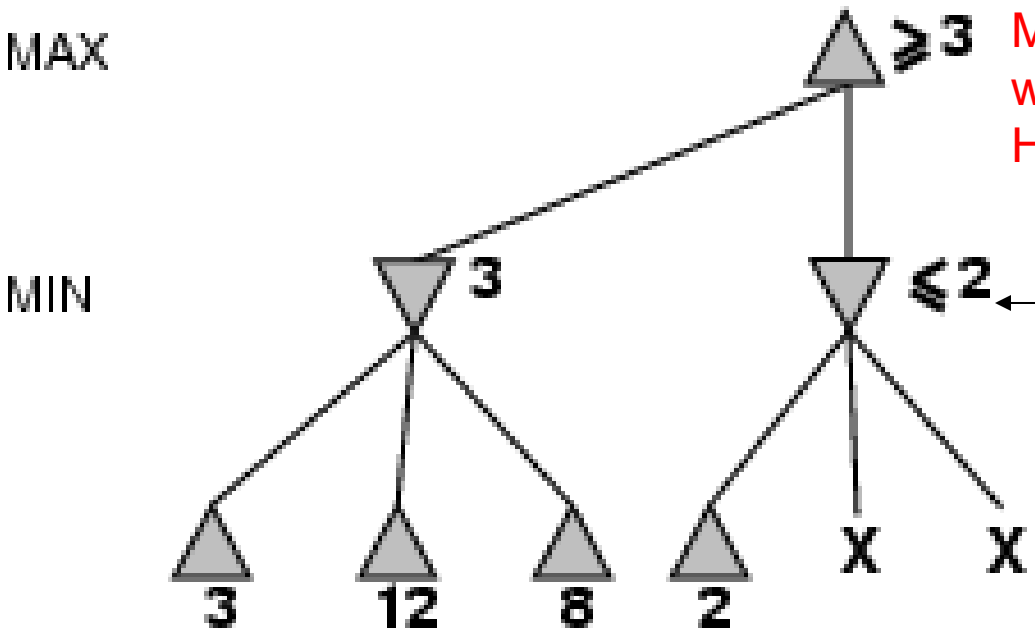
MAX knows that it can at least get "3" by playing this branch

3

MIN will choose "3", because it minimizes the utility (which is good for MIN)

3 12 8

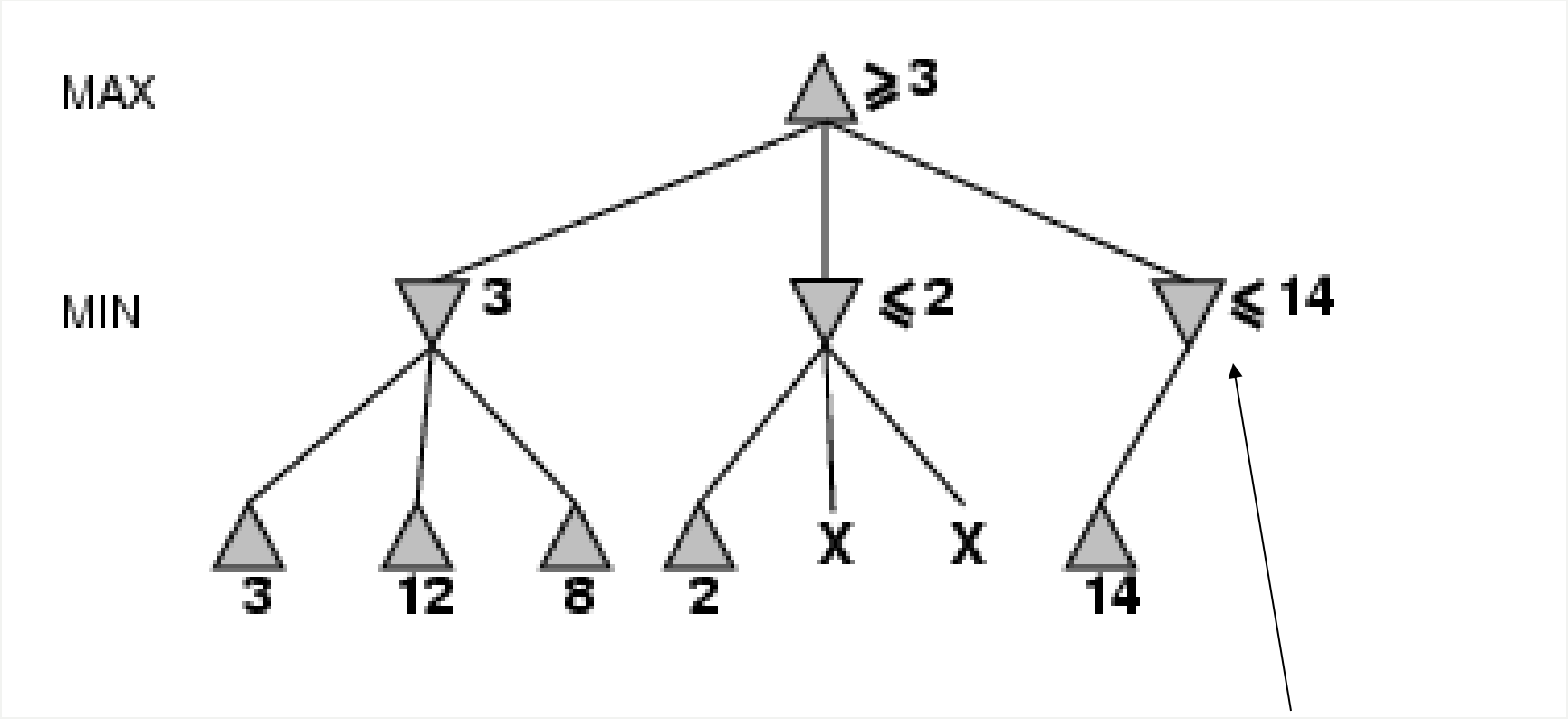
α-β pruning example



MAX knows that the new branch will never be better than 2 for him. He can *ignore* it.

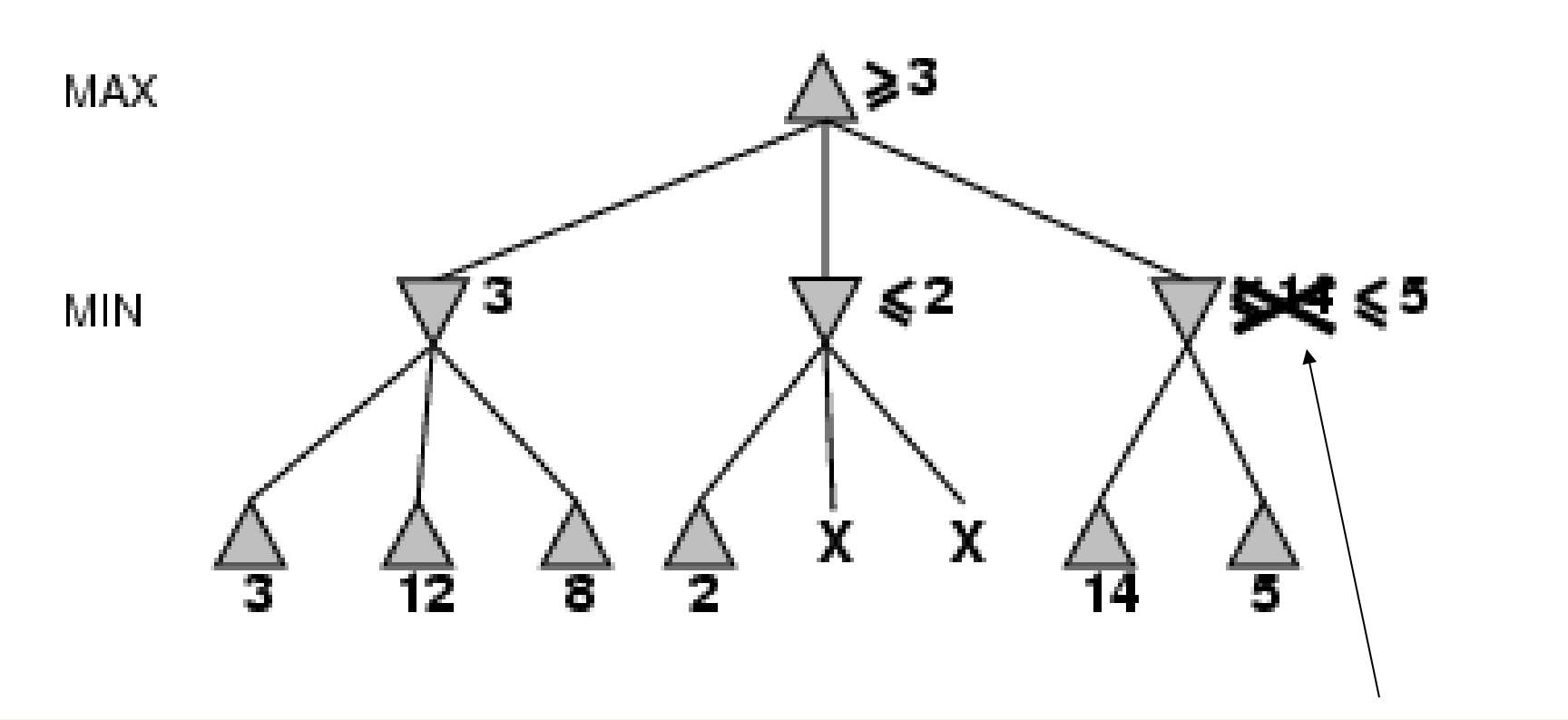
MIN can certainly do as good as 2, but maybe better (= smaller)

α-β pruning example



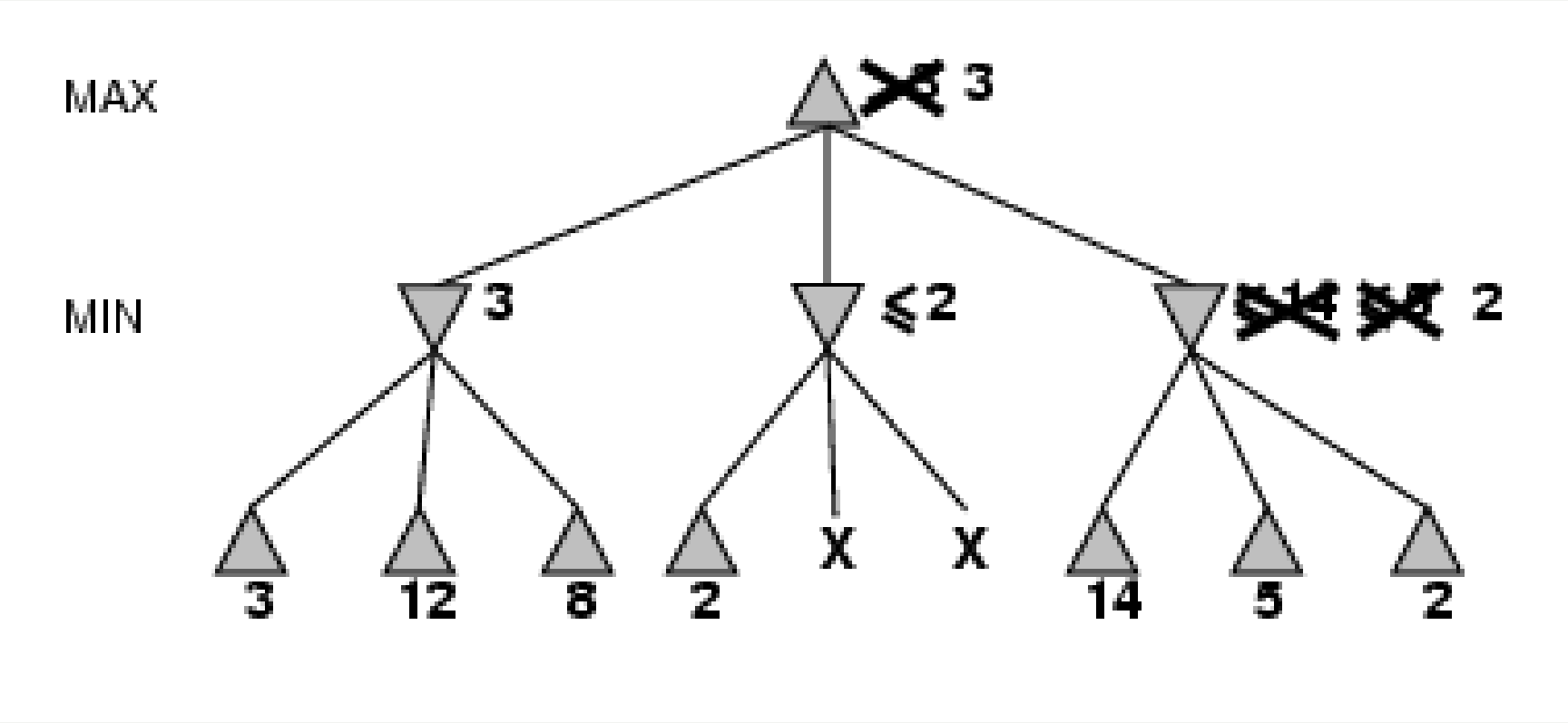
MIN will do at least as good as 14 in this branch (which is very good for MAX!) so MAX will want to explore this branch more.

α-β pruning example

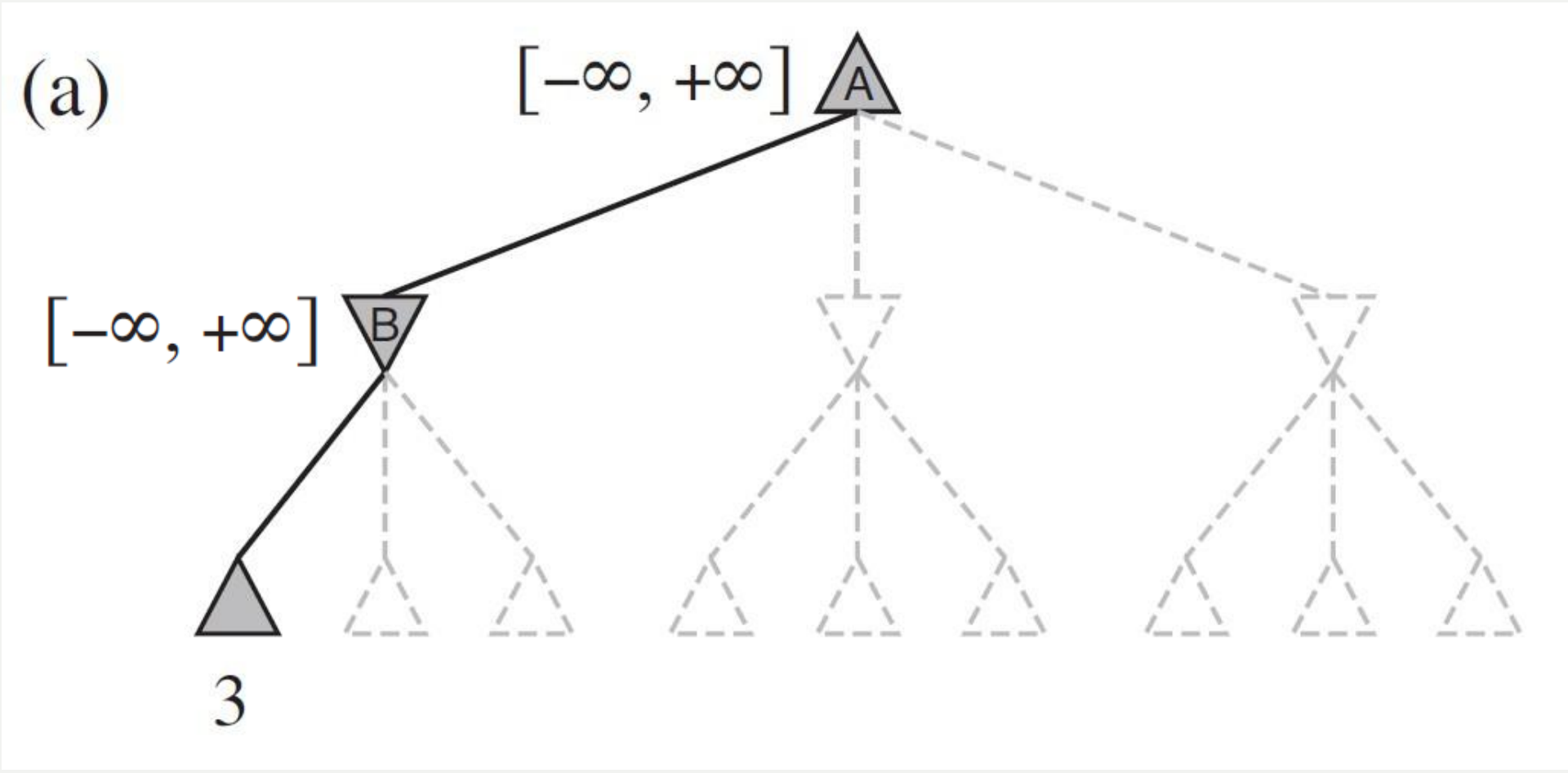


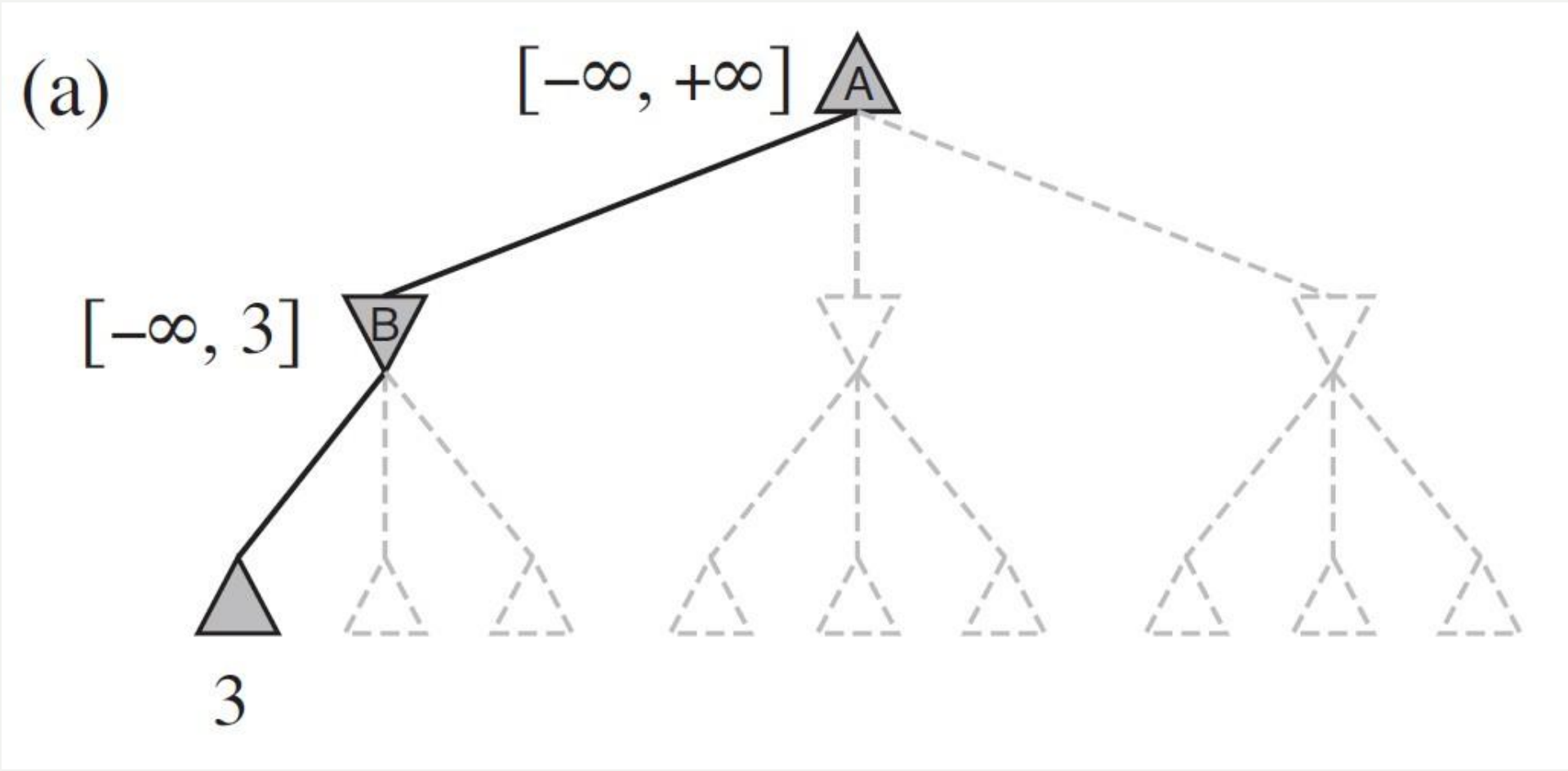
MIN will do at least as good as 5 in this branch (which is still good for MAX) so MAX will want to explore this branch more.

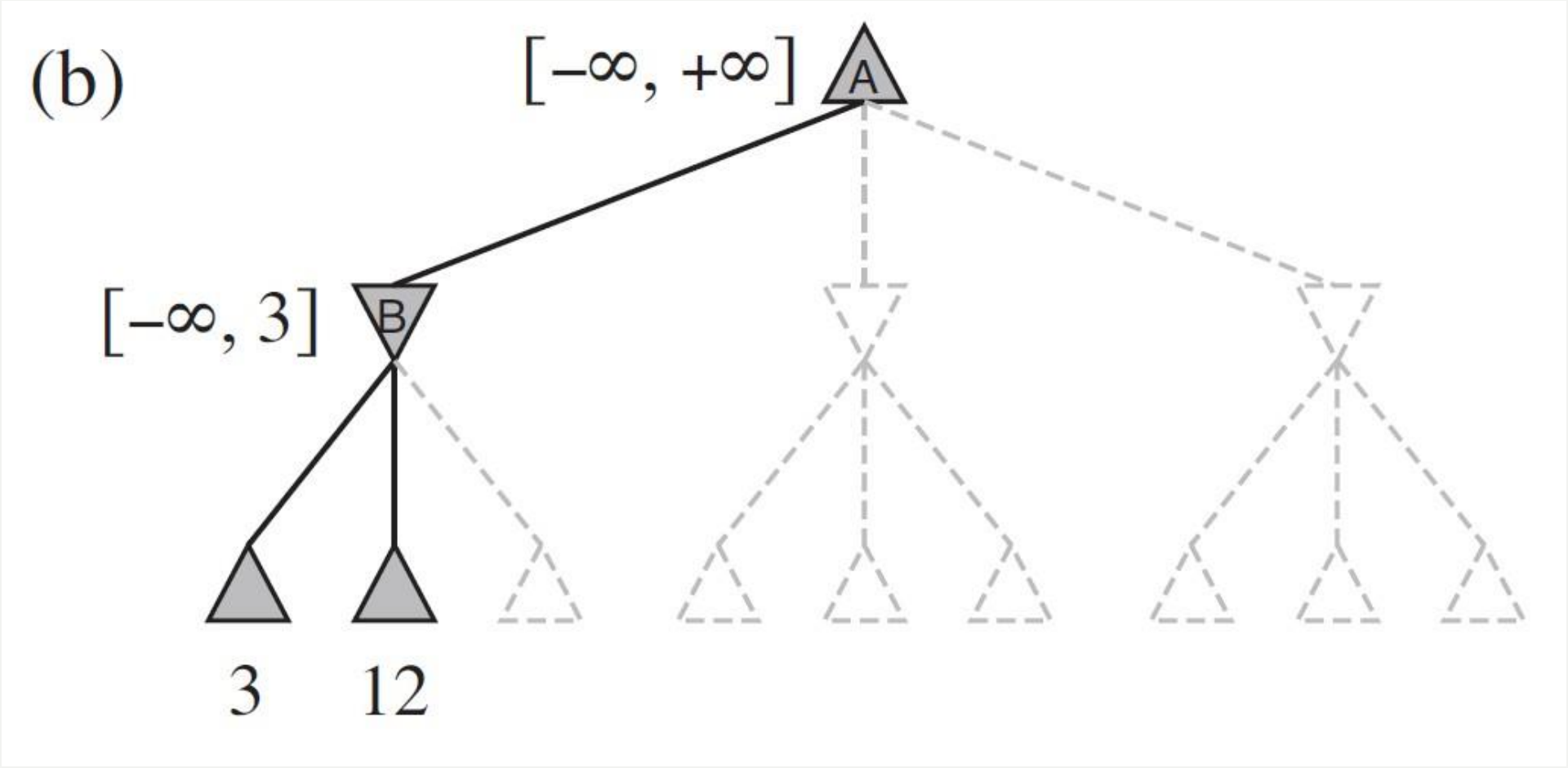
α-β pruning example

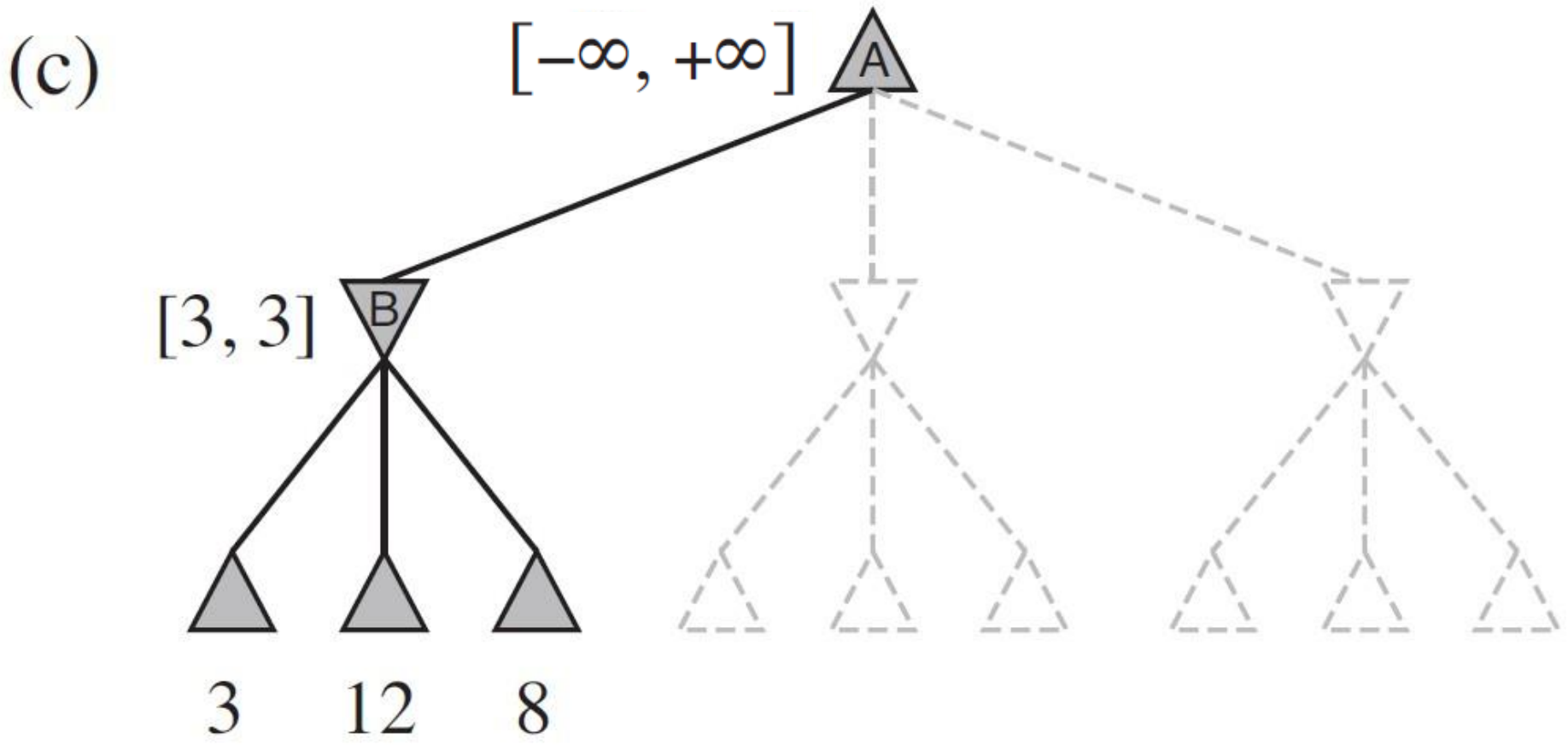


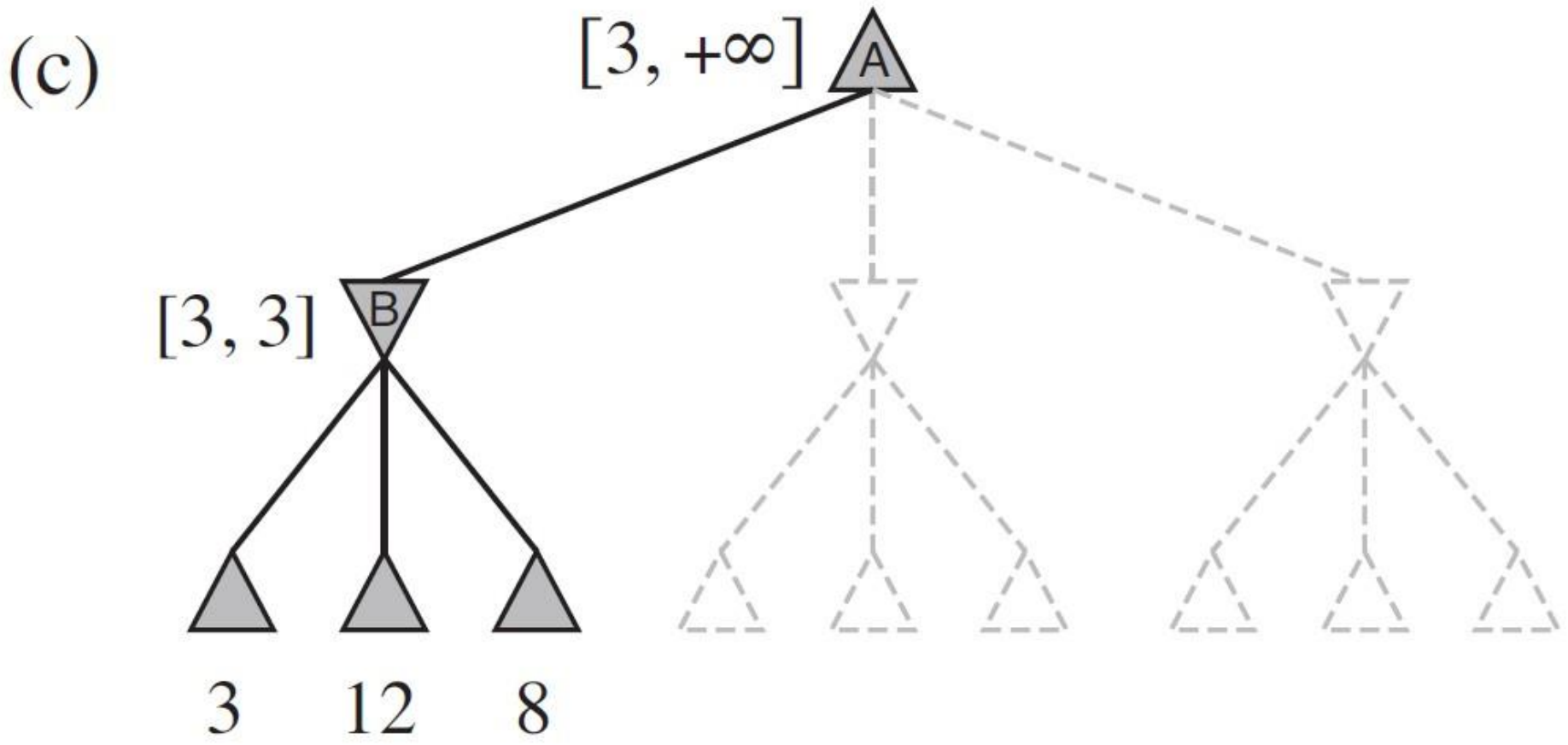
Bummer (for MAX): MIN will be able to play this last branch and get 2. This is worse than 3, so MAX will play 3.

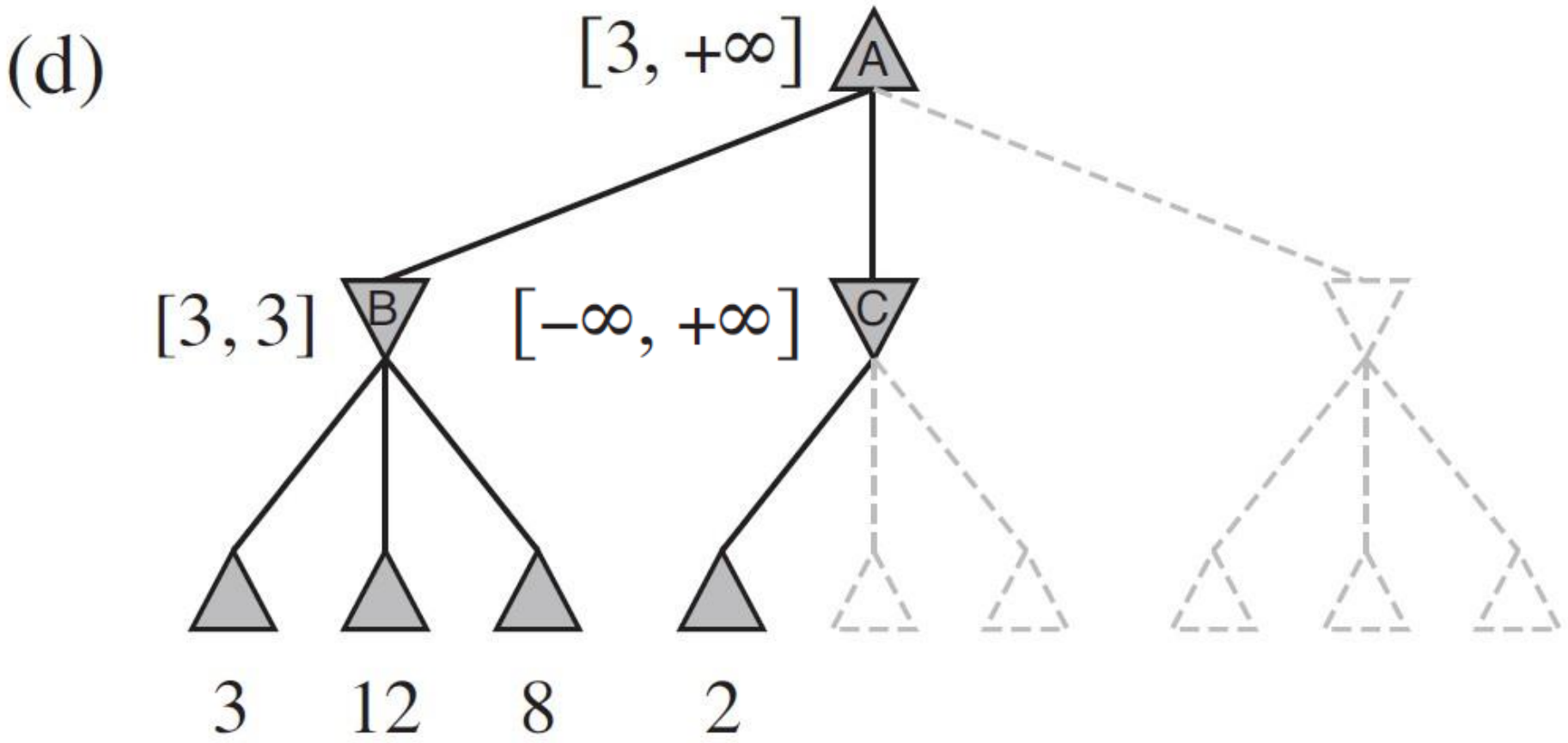

(a)
[−∞, +∞]
A
[−∞, +∞]
B
3

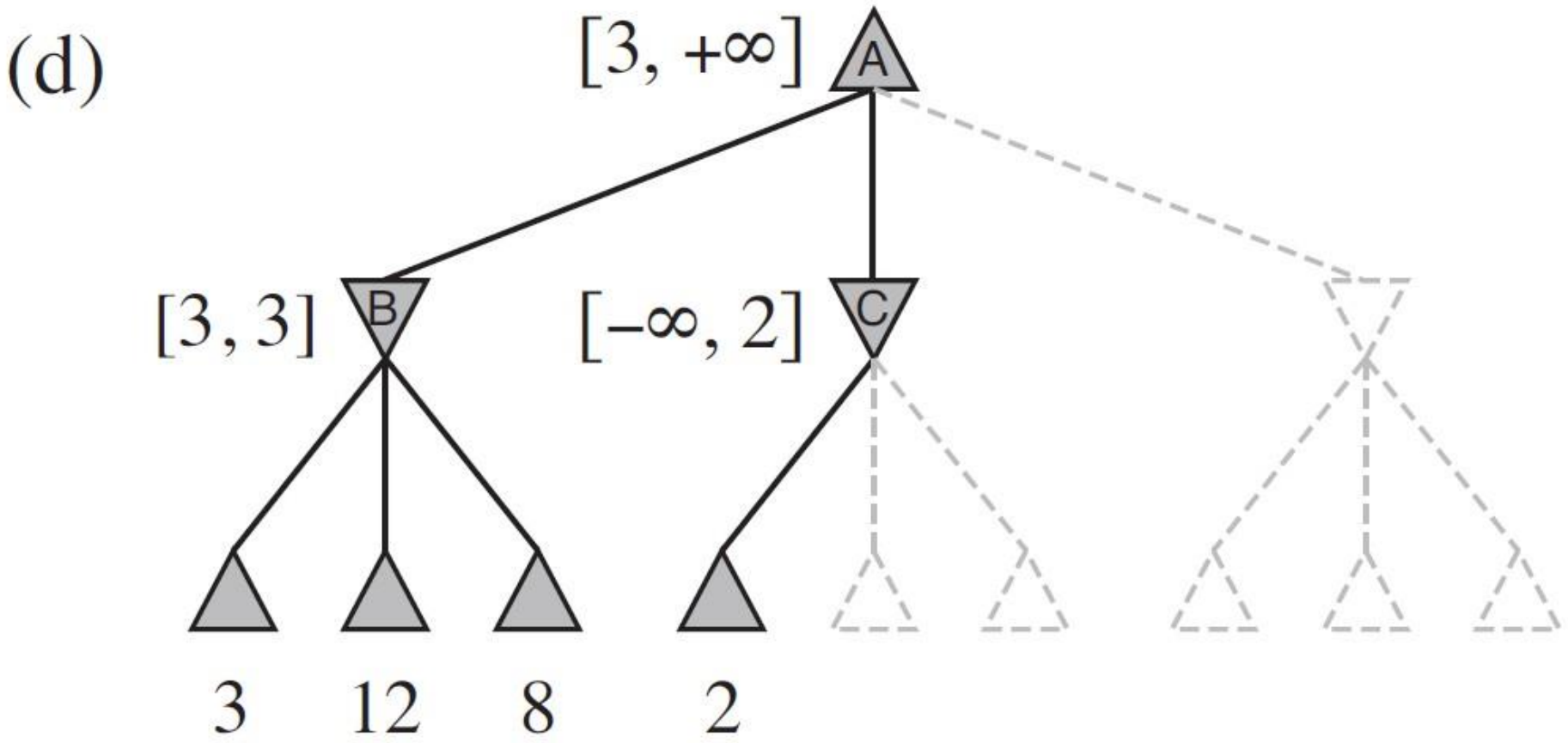

(a)
[−∞, +∞]
A
[−∞, 3]
B
3

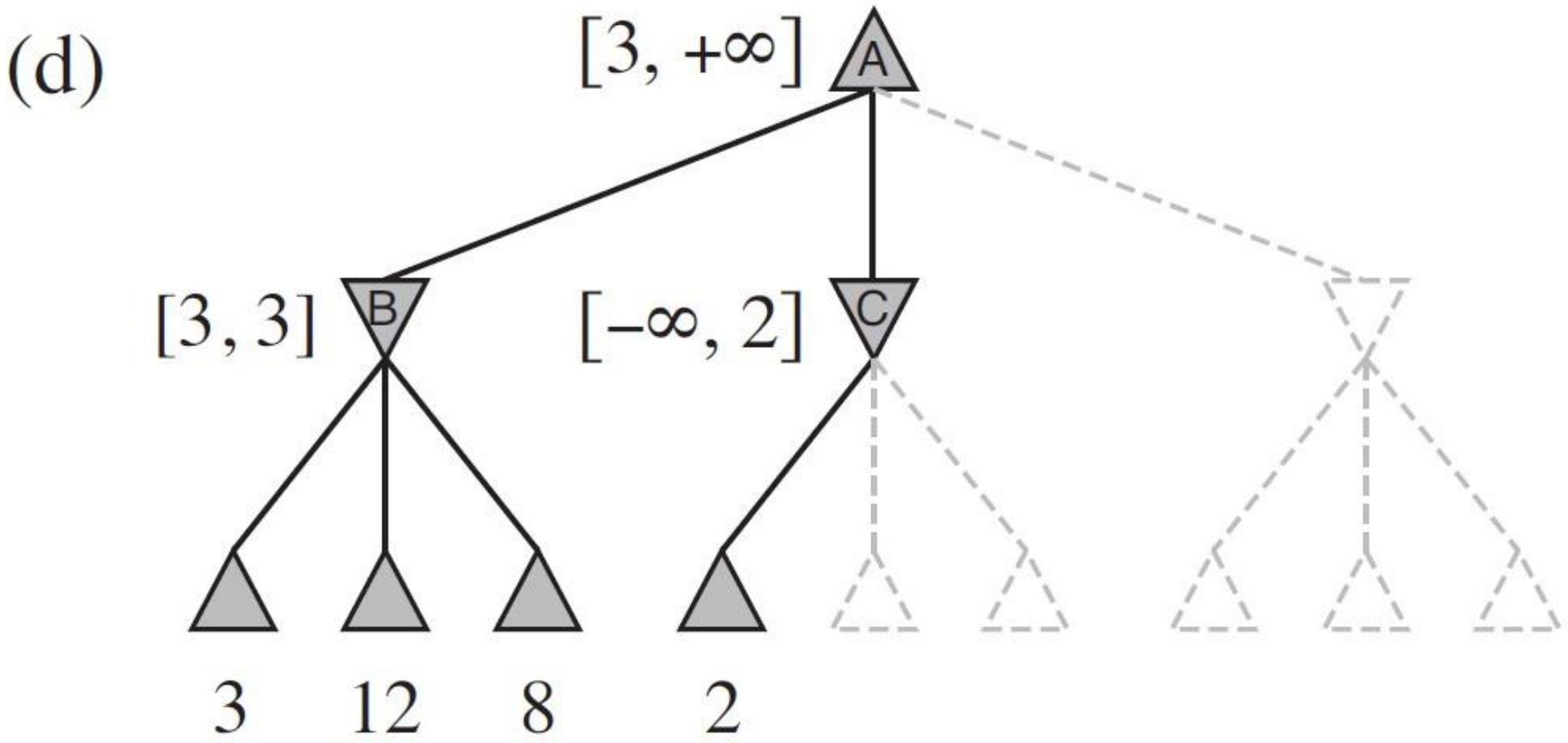

(b)
[−∞, +∞]
A
[−∞, 3]
B
3
12

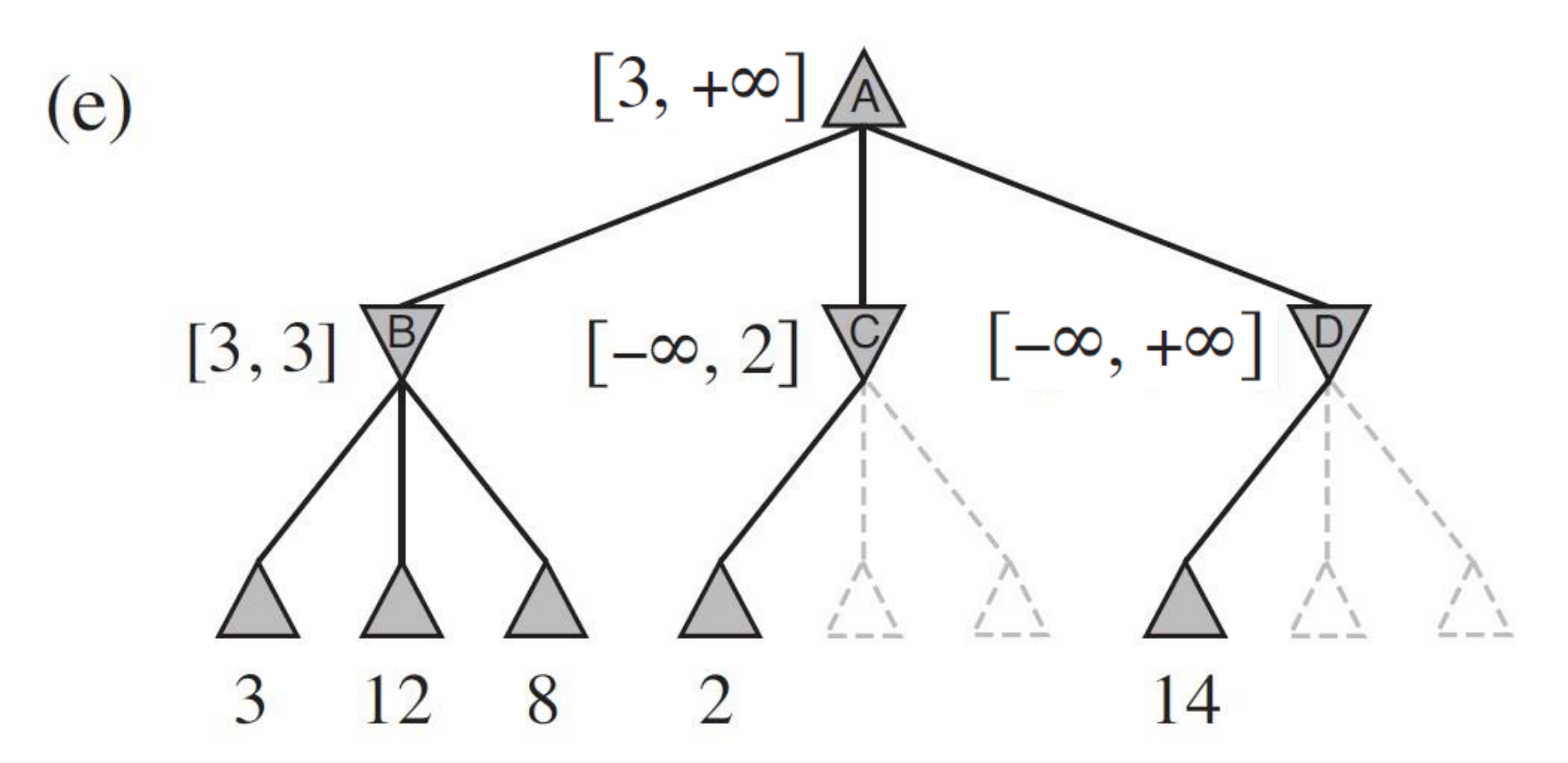

(c)
[−∞, +∞]
A
[3, 3]
B
3
12
8

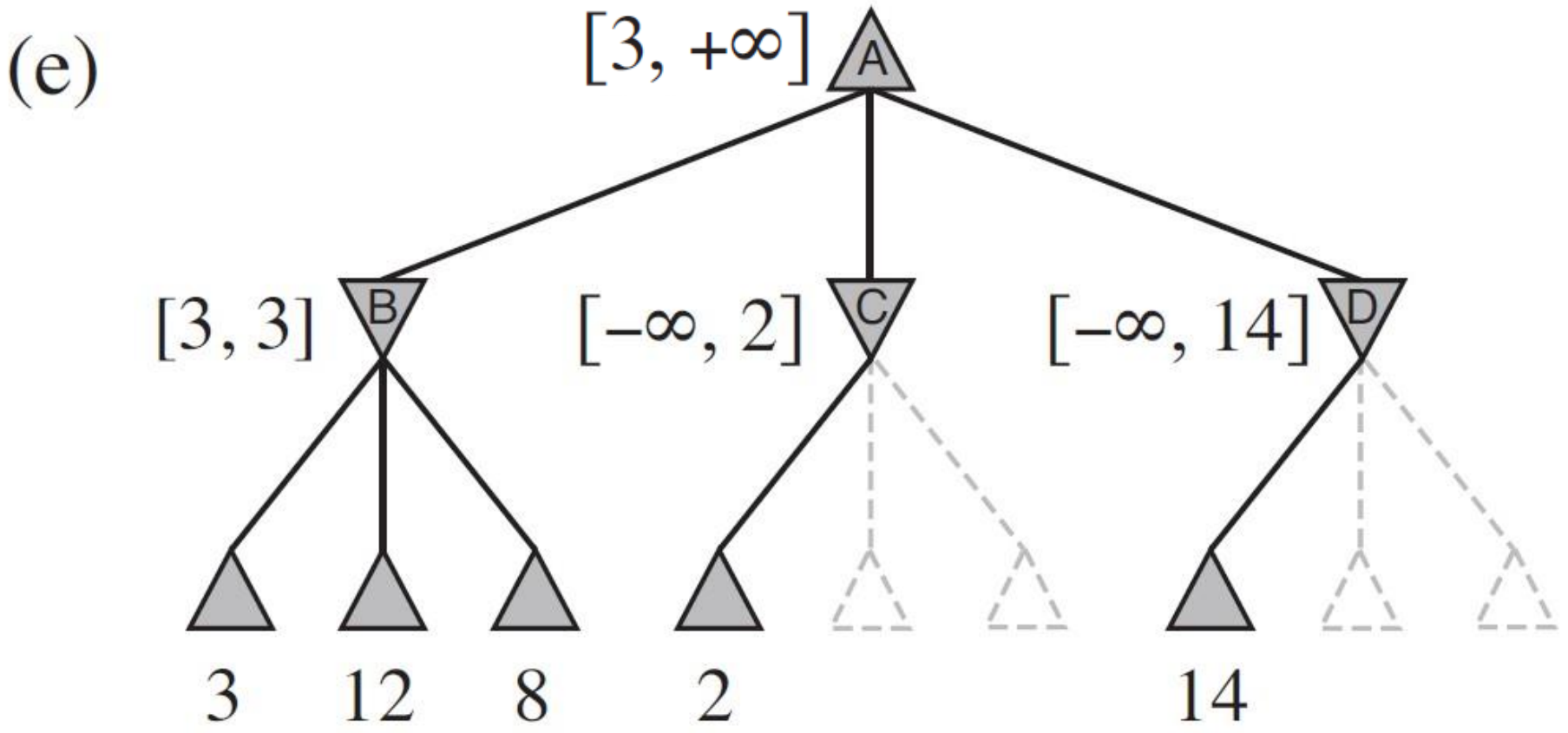

(c)
[3, +∞]
A
[3, 3]
B
3
12
8

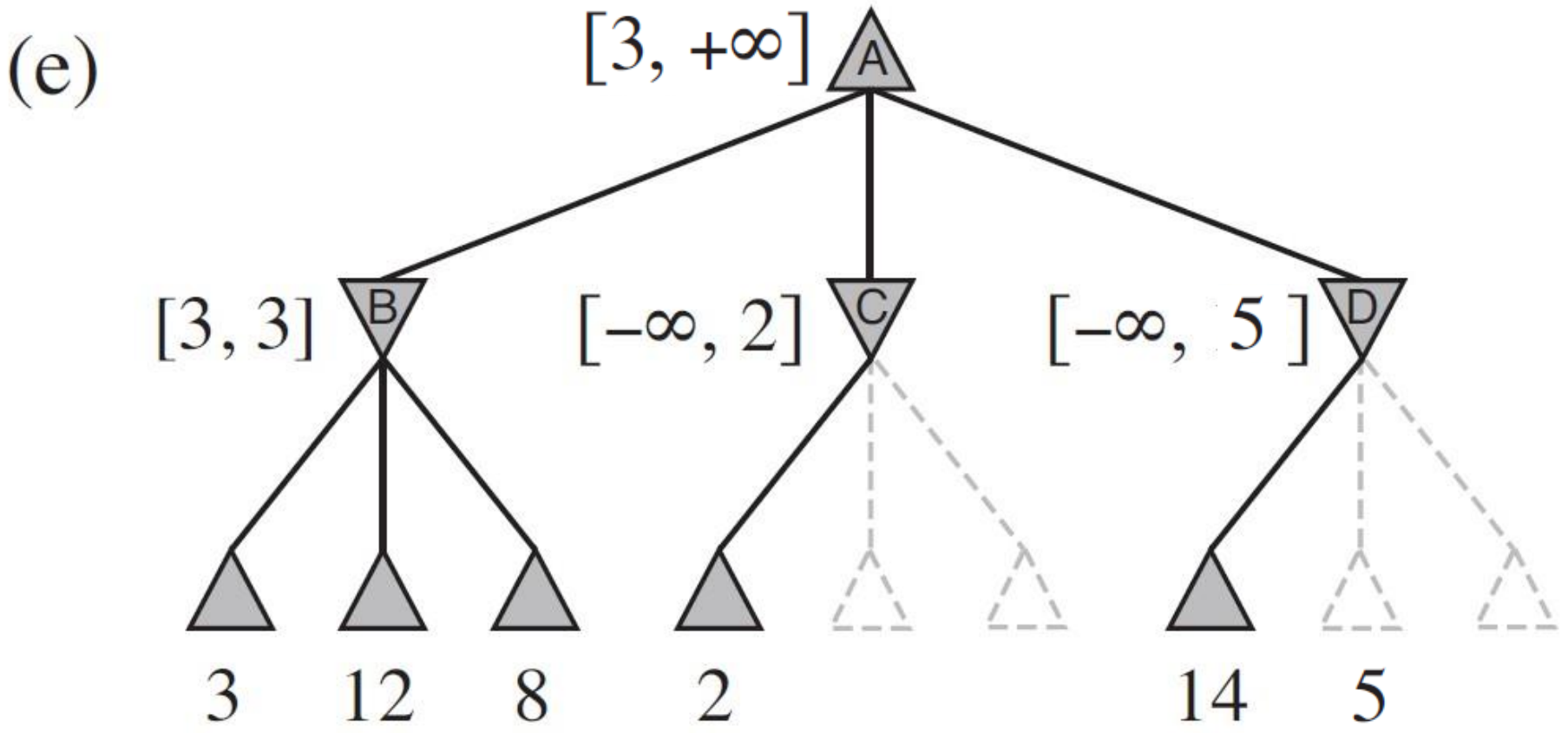

(d)
[3, +∞]
A
[3, 3]
B
[−∞, +∞]
C
3
12
8
2

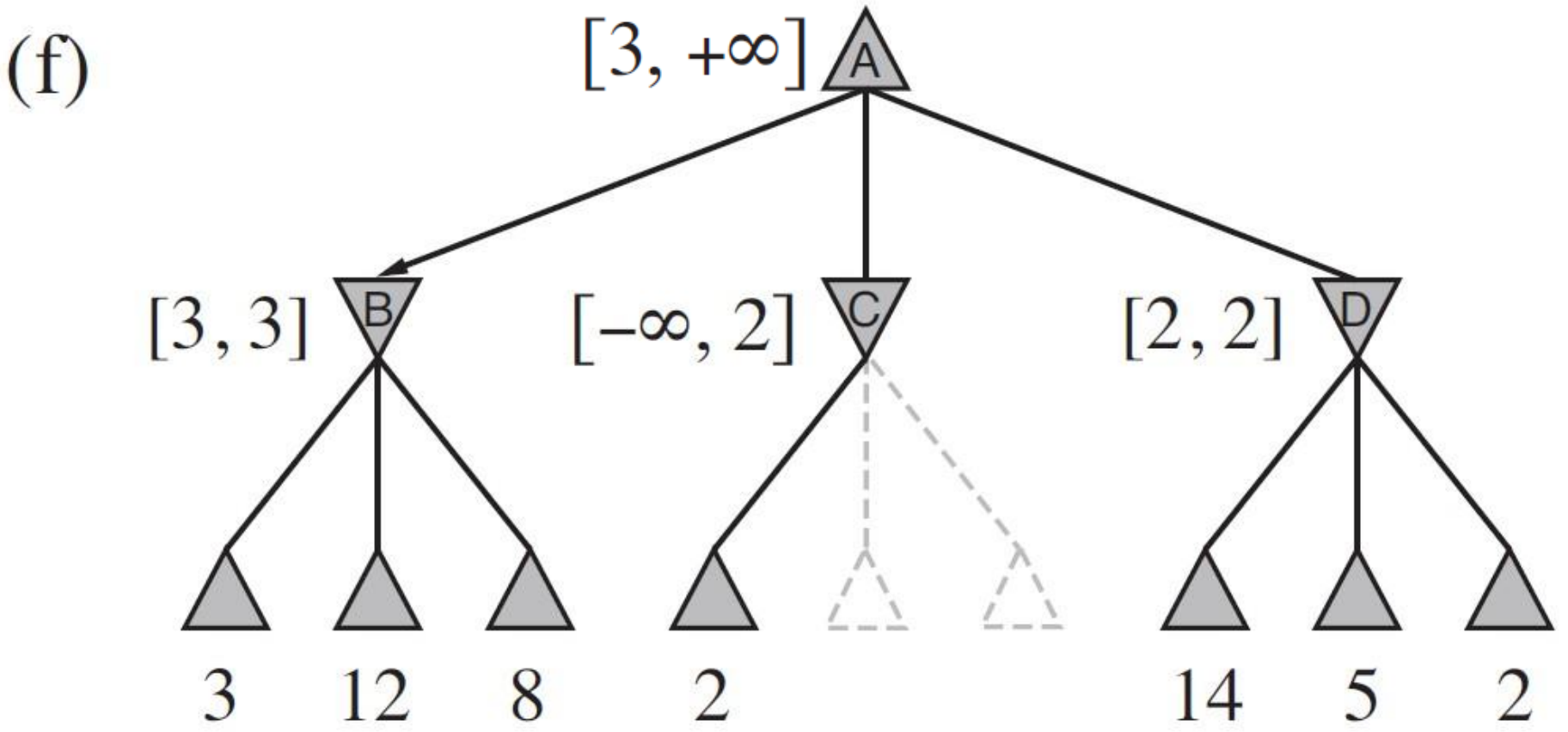

(d)
[3, +∞]
A
[3, 3]
B
[−∞, 2]
C
3
12
8
2

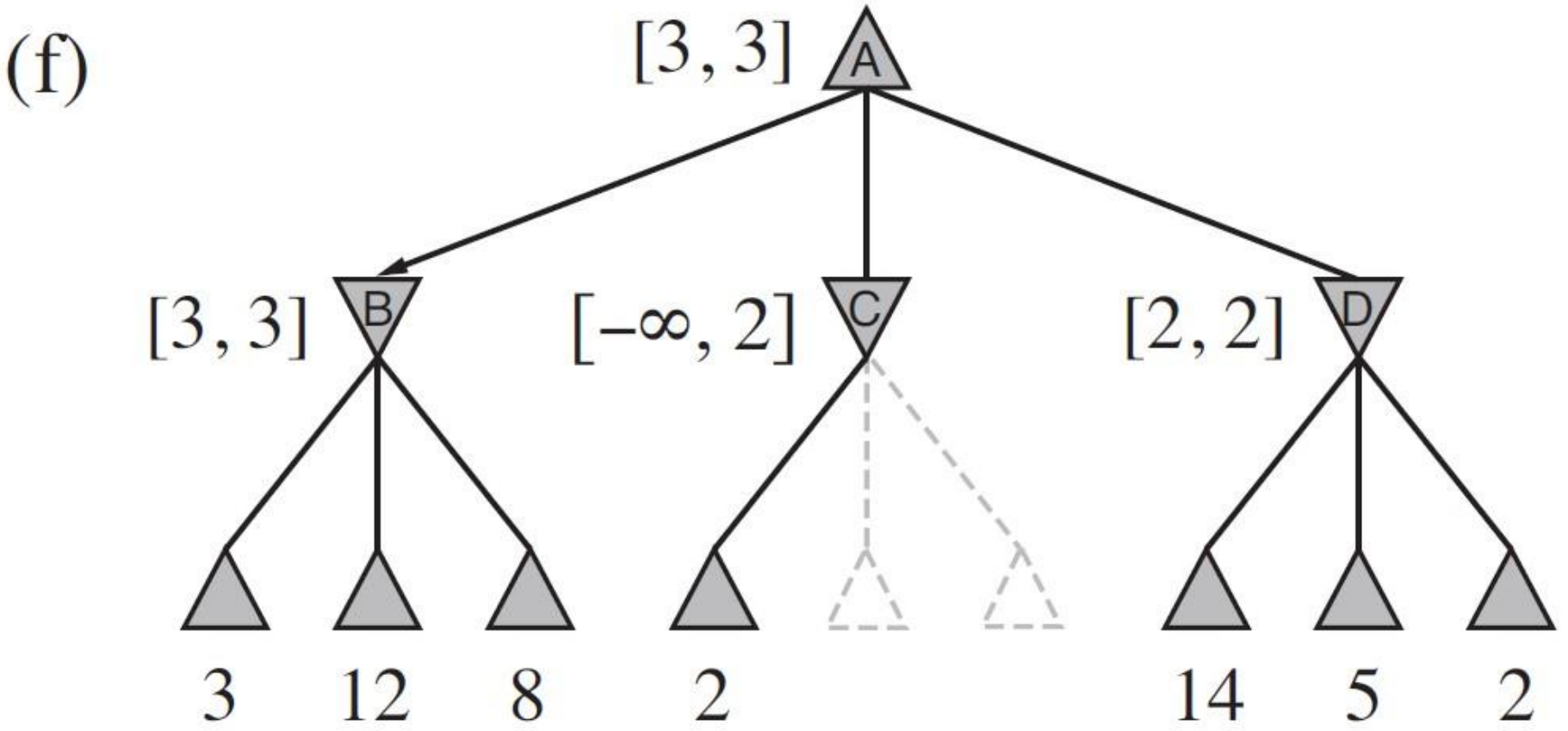

(d)
[3, +∞]
A
[3, 3]
B
[−∞, 2]
C
3
12
8
2


(e)
[3, +∞]
A
[3, 3]
B
[−∞, 2]
C
[−∞, +∞]
D
3
12
8
2
14


(e)
[3, +∞]
A
[3, 3]
B
[−∞, 2]
C
[−∞, 14]
D
3
12
8
2
14

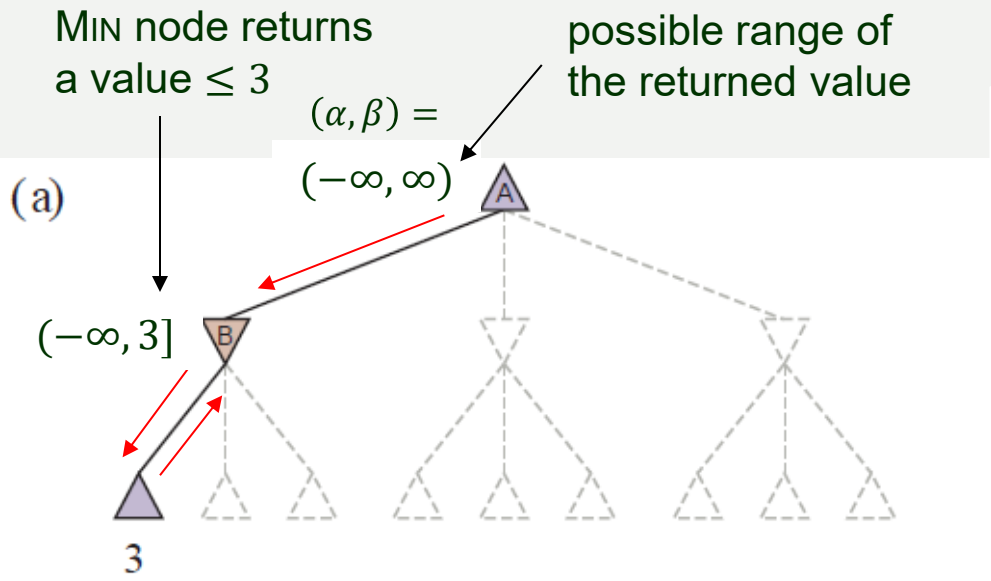

(e)
[3, +∞]
A
[3, 3]
B
[−∞, 2]
C
[−∞, 5]
D
3
12
8
2
14
5

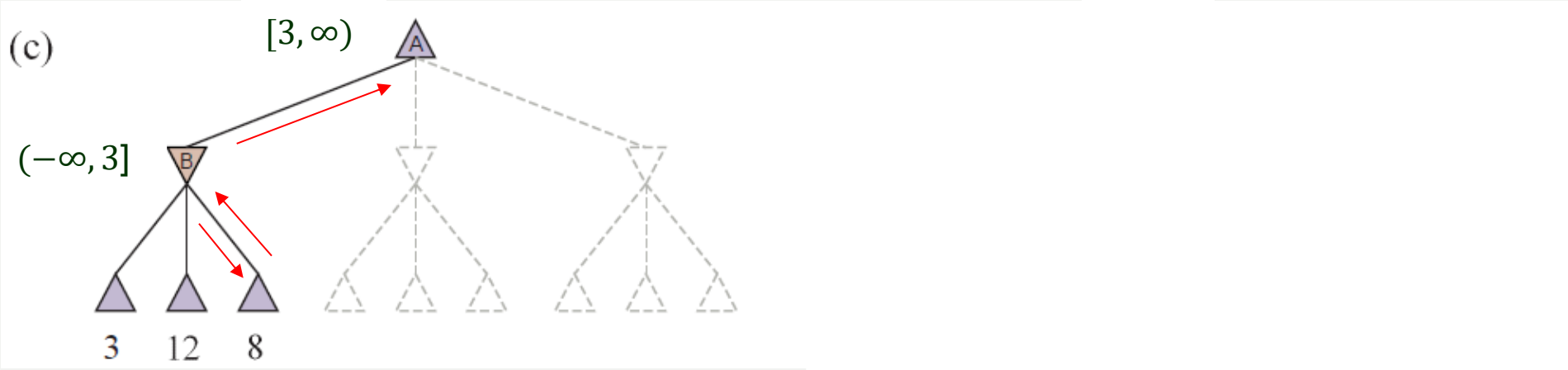

(f)
[3, +∞]
A
[3, 3]
B
[−∞, 2]
C
[2, 2]
D
3
12
8
2
14
5
2


(f)
[3, 3]
A
[3, 3]
B
[−∞, 2]
C
[2, 2]
D
3
12
8
2
14
5
2

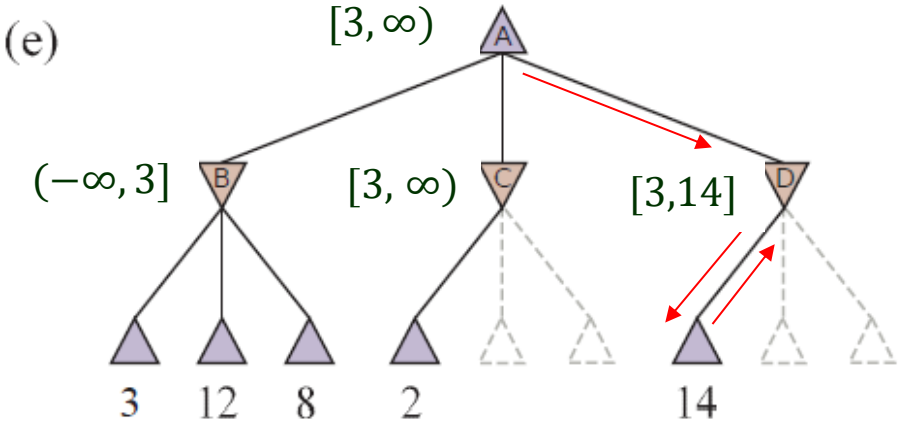
Stages in the calculation of the optimal decision for the game tree in Figure 5.2. At each point, we show the range of possible values for each node. (a) The first leaf below B has the value 3. Hence, B, which is a MIN node, has a value of *at most* 3. (b) The second leaf below B has a value of 12; MIN would avoid this move, so the value of B is still at most 3. (c) The third leaf below B has a value of 8; we have seen all B's successor states, so the value of B is exactly 3. Now, we can infer that the value of the root is *at least* 3, because MAX has a choice worth 3 at the root. (d) The first leaf below C has the value 2. Hence, C, which is a MIN node, has a value of *at most* 2. But we know that B is worth 3, so MAX would never choose C. Therefore, there is no point in looking at the other successor states of C. This is an example of alpha–beta pruning. (e) The first leaf below D has the value 14, so D is worth *at most* 14. This is still higher than MAX's best alternative (i.e., 3), so we need to keep exploring D's successor states. Notice also that we now have bounds on all of the successors of the root, so the root's value is also at most 14. (f) The second successor of D is worth 5, so again we need to keep exploring. The third successor is worth 2, so now D is worth exactly 2. MAX's decision at the root is to move to B, giving a value of 3.

Fig. 5.5 in the 4th edition of the textbook *incorrectly executes* the algorithm ALPHA-BETA-SEARCH in Fig. 5.7.

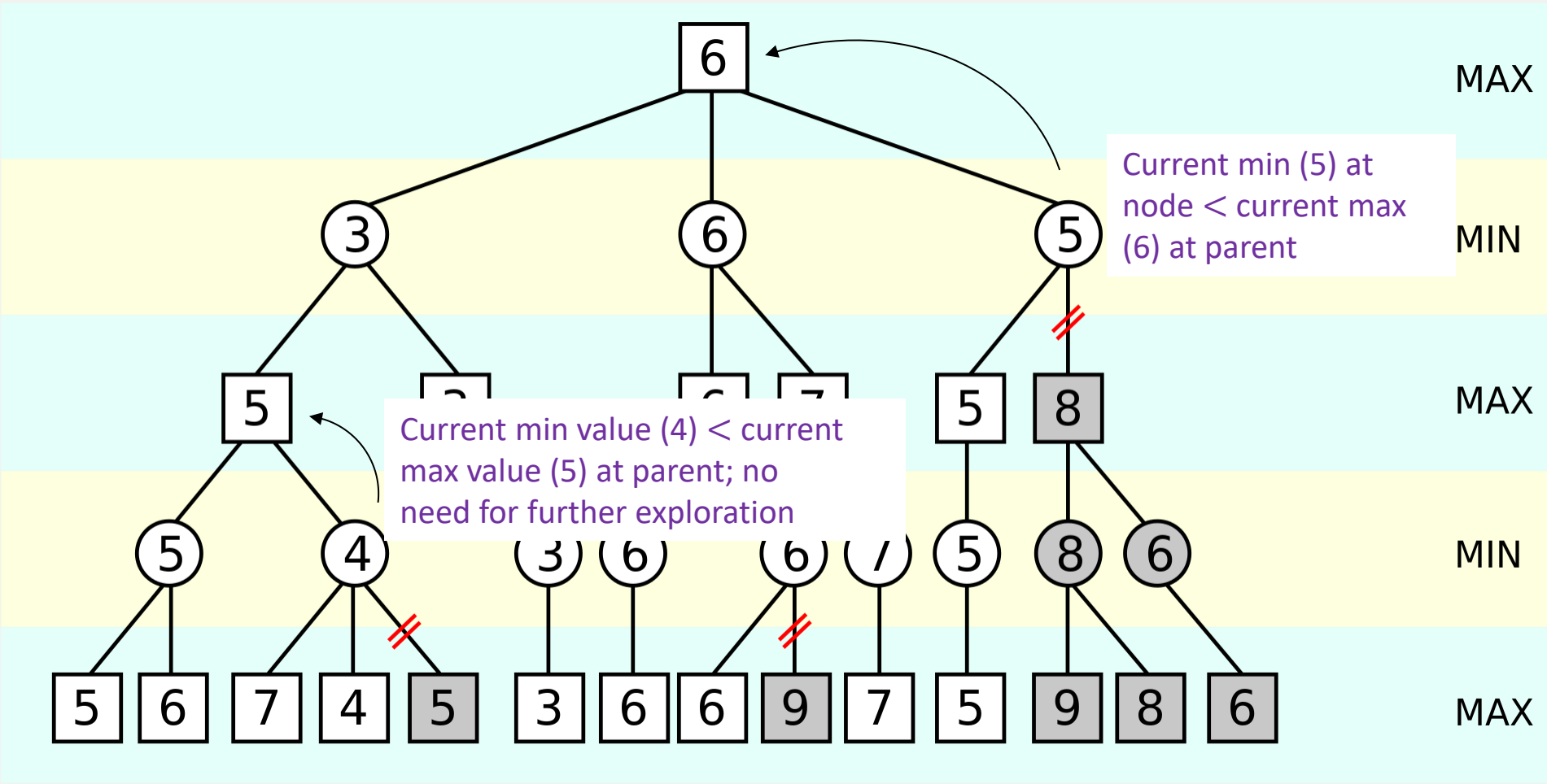



(c)
[3, ∞)
A
(−∞, 3]
B
3
12
8

MINIMAX(root) $= \max(\min(3,12,8), \min(2, x, y), \min(14, 5, 2))$
$= \max(3, \min(2, x, y), 2)$
$= \max(3, z, 2)$ where $z = \min(2, x, y) \leq 2$
$= 3.$



A Larger Example (Wikipedia)

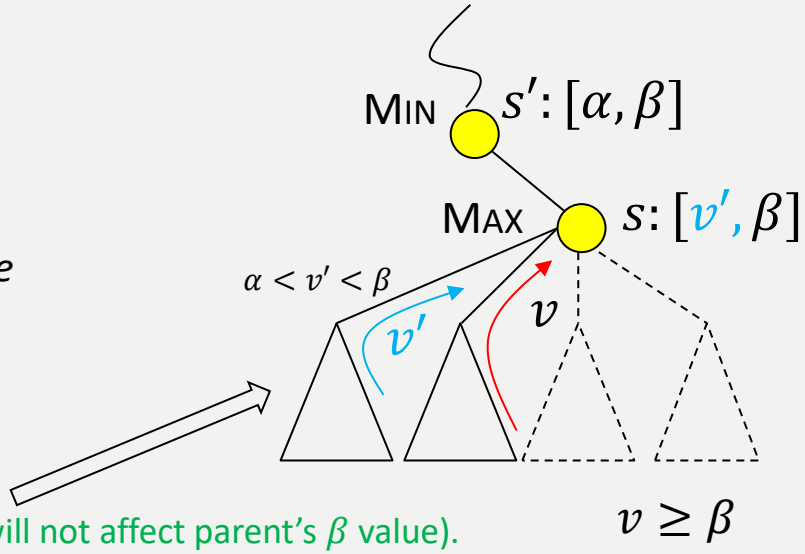


Exercise (suggested): Trace the execution of ALPHA-BETA-SEARCH (next page) and dynamically update the $[\alpha, \beta]$ value of every expanded node.

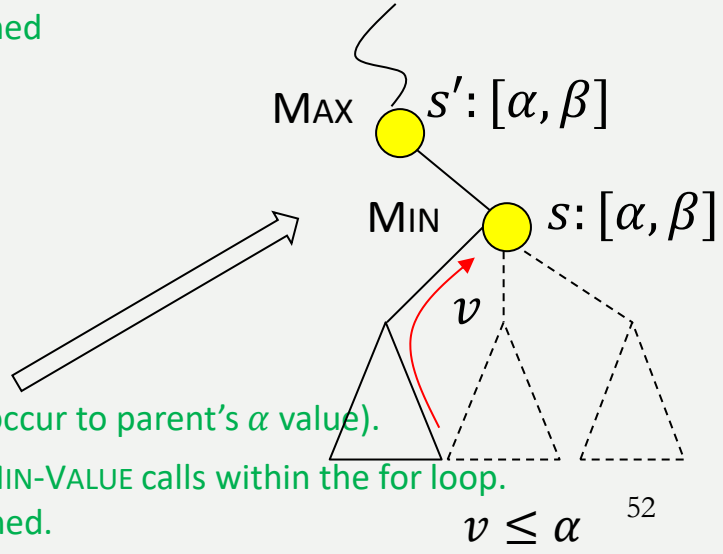
Alpha-Beta Search Algorithm (3rd Edition)

function ALPHA-BETA-SEARCH(*state*) **returns** an action
  $v \leftarrow$MAX-VALUE(*state*, $-\infty, +\infty$)
  return the action in ACTIONS(*state*) with value v

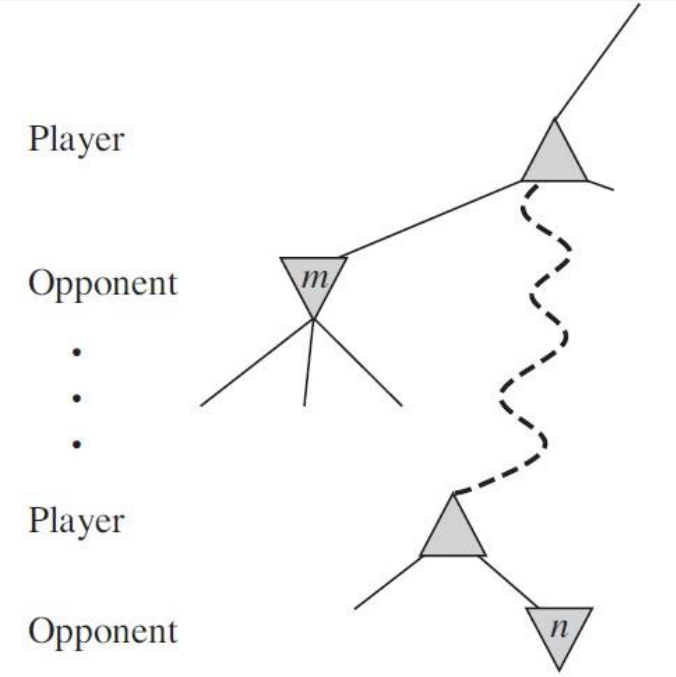
// no change of β value within MAX-VALUE()
function MAX-VALUE (*state*,α, β) **returns** *a utility value*
  if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
  $v \leftarrow -\infty$
  for each a in ACTIONS(*state*) **do**
    $v \leftarrow$MAX(v, MIN-VALUE(RESULT(s, a),α, β))
    if $v \geq \beta$ **then return** v // pruning (returned v will not affect parent's β value).
    $\alpha \leftarrow$MAX(α, v) // $v < \beta$: new α passed on to the remaining MIN-VALUE calls within the for loop.
  return v // maximum value of all children if none are pruned



// no change of α value within MIN-VALUE()
function MIN-VALUE (*state*,α, β) **returns** *a utility value*
  if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
  $v \leftarrow +\infty$
  for each a in ACTIONS(*state*) **do**
    $v \leftarrow$MIN(v, MAX-VALUE(RESULT(s, a),α, β))
    if $v \leq \alpha$ **then return** v // pruning (no change will occur to parent's α value).
    $\beta \leftarrow$MIN(β, v) // $v > \alpha$: new β passed on to the rest MIN-VALUE calls within the for loop.
  return v // minimum value of all children if none are pruned.



- Alpha-beta pruning can be applied to trees of any depth
  - Can prune the entire subtree



Properties of alpha-beta search

- Alpha-beta produces the same optimal move as minimax
  - Alpha-beta is an exact search, not an approximate search
- Good ordering:
  - Action with a higher utility for MAX (and a lower utility for MIN) comes first
  - It can improve effectiveness of pruning

- With 'perfect ordering', time complexity = $O(b^{m/2})$
  - Can search twice as deep for the same time amount
  - Effect branching factor is $b^{1/2}$
  - Unfortunately, 35^{50} is still infeasible for chess

4/7. Imperfect real-time decisions

- Resources are limited in reality: time, memory, etc
  - chess: you're expected to make a move in a few minutes
- Heuristic Minimax (Claude Shannon, 1950)
  - Make a decision using
    - cutoff test (instead of terminal test)
    - heuristic evaluation function (instead of utility)

Cut off the search early by applying a heuristic evaluation function

1)Replace the utility function by a heuristic evaluation function EVAL, which estimates the position's utility. 2) replace the terminal test by a cutoff test that decides when to apply EVAL.
: s (state), d (maximum depth)

$$\text{H-MINIMAX}(s,d) = \begin{cases} \text{EVAL}(s) & \text{if CUTOFF-TEST}(s,d) \\ \max_{a \in Actions(s)} \text{H-MINIMAX}(\text{RESULT}(s,a), d+1) & \text{if PLAYER}(s) = \text{MAX} \\ \min_{a \in Actions(s)} \text{H-MINIMAX}(\text{RESULT}(s,a), d+1) & \text{if PLAYER}(s) = \text{MIN}. \end{cases}$$

Evaluation function

- Conditions
  - Should be easy to compute
  - In terminal states: eval(“win”) > eval(“draw”) > eval(“loss”)
  - In non-terminal states, good eval should correlate strongly with the actual chance of winning
- Example 1. expected utility from past records
  - In two-pawn vs one-pawn situation
    - 72% win, 20% loss, 8% draw
    - Utility: 1 win, 1/2 draw, 0 loss
    - Expected utility = 0.72 x 1 + 0.08 x ½ + 0.20 x 0 = 0.76

- Example 2. features
  - In chess
    - Define material values of pieces in chess:
      Pawn(1), knight/bishop(3), rook(5), queen(9), etc
    - Compute the current total value by summing
      1 x # of Pawns + 3 x # of knights/bishops + ...
  - More generally
    - $f_i(s)$ is a feature of the state s (# of pieces)
    - w_i : value of i-th feature

$$\textsc{Eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \cdots + w_n f_n(s) = \sum_{i=1}^{n} w_i f_i(s) \ ,$$

Cutting off

- Instead of the terminal test in alpha-beta, use

  i.e., return if it exceeds the depth cut-off
- How to handle time cut-off? (e.g., 3 mins)
  - Iterative deepening search:
    - search with depth cut-offs of 1, 2, 3, …..
    - When time runs out, return the best move from the deepest completed search
- Simple cutoff by depth limit does not always work

More tricks for chess

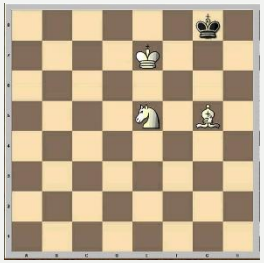
- Forward pruning
  - PROBCUT (1995): Alpha-beta search prunes any node that is provably and probably outside (alpha,beta)
- Putting these together for PC chess agent
  - Suppose we can generate/evaluate 200 million nodes in three minutes
  - Minimax → 5 plies (35^5 = 50 mil)
  - Alpha-beta → 10 plies
  - Other tricks → 14 plies

The branching factor for chess is about 35, on average, and 35^5 is about 50 million, so if we used minimax, we could look ahead only about 5 plies,

Search vs lookup

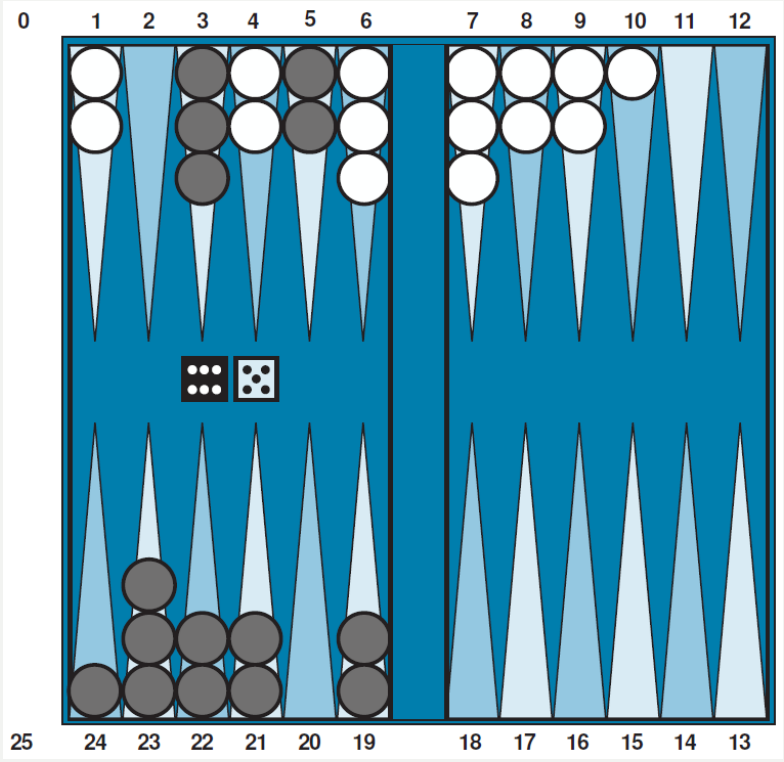
Near the end of the game there are fewer possible positions, and thus more chance to do lookup; A computer can completely solve the endgame by producing policy, which is a mapping from every possible state to the best move in that state

- Look-up approach
  - Good opening and endgames are often known
  - Create lookup tables from, human expertise or a database of previously played games
- Switch between search and lookup
- Endgames
  - King-bishop-knight-versus-king: (3,494,568) positions
  - All endgames with up to five pieces and some six pieces
  - Checkers: all endgames known



5/7. Stochastic games

Some games (e.g., backgammon) have randomness due to throwing of dice. They combine both luck and skill.



- Include *chance nodes* in addition to MAX and MIN nodes.

  Throwing two dice (unordered):

  1-1,…,6-6: probability $1/36$ each

  1-2,…,1-6, 2-3,…, 5-6: probability $1/18$ each

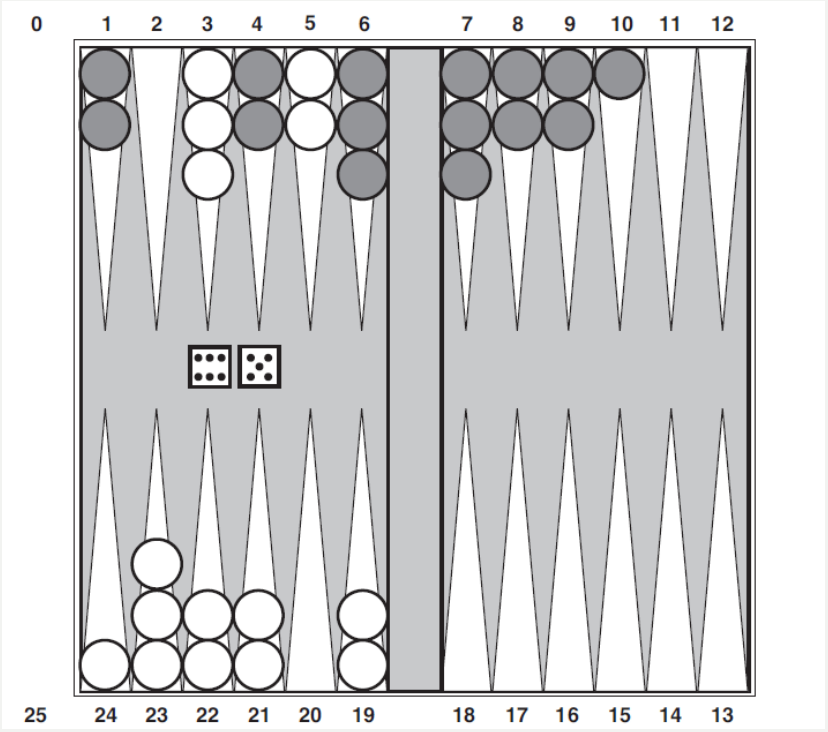
  15

- Calculate expected value (called *expectiminimax* value) of a position.

Games incorporating unpredictability by including a random element, s.t the throwing of dice

- Backgammon
  - White: clock-wise
  - Black: counter-clock
  - Video demo (https://www.youtube.com/watch?v=xXE5AwzNQ2s)

62



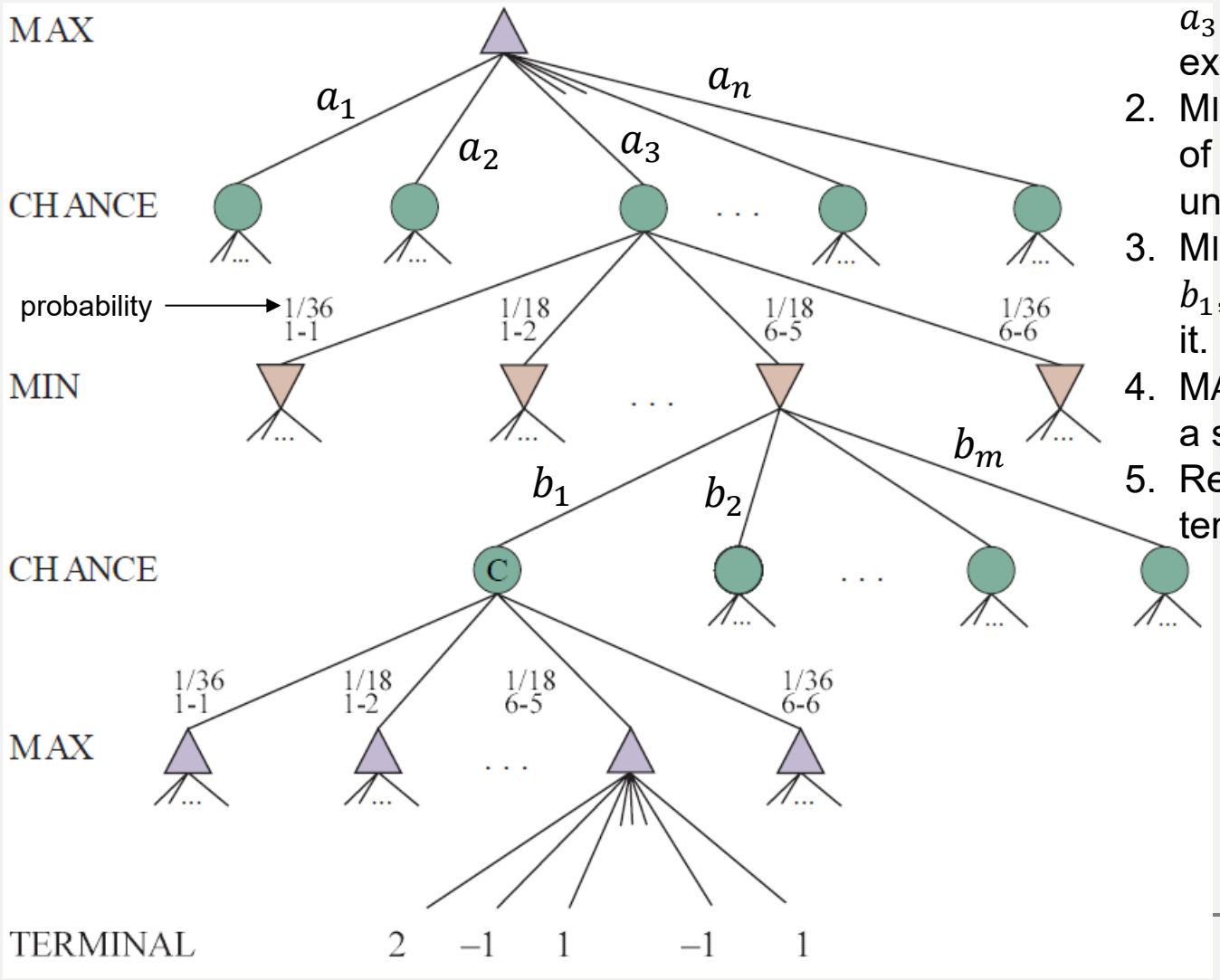
A typical backgammon position. The goal of the game is to move all one's pieces off the board. White moves clockwise toward 25, and Black moves counterclockwise toward 0. A piece can move to any position unless multiple opponent pieces are there; if there is one opponent, it is captured and must start over. In the position shown, White has rolled 6–5 and must choose among four legal moves: (5–10,5–11), (5–11,19–24), (5–10,10–16),and (5–11,11–16), where the notation (5–11,11–16) means move one piece from position 5 to 11, and then move a piece from 11 to 16.

- White: 6 and 5
  - (5-10,5-11), (5-11,19-24), (5-10,10-16), (5-11,11-16)

- After dice are rolled, White knows possible actions
- White does not know what Black is going to roll and what action it will choose
- CHANCE NODES are required in the game tree
  - It's a game search version of AND-OR search

The branch leading from each chance node denote the possible dice rolls; each branch is labeled with the roll and its probability. There are 36 ways to roll two dice, each equally likely; but because a 6–5 is the same as a 5–6, there are only 21 distinct rolls. The six doubles (1–1 through 6–6) each have a probability of 1/36, so we say P(1–1) = 1/36. The other 15 distinct rolls each have a 1/18 probability

Stochastic game trees



1. MAX chooses an action, say, a_3 out of $\{a_1,..,a_n\}$ and executes it.
2. MIN rolls a die and gets a set of legal actions $\{b_1,..,b_m\}$ under some CHANCE node.
3. MIN chooses an action, say, b_1, from the set and executes it.
4. MAX rolls a die and gets a set of legal actions.
5. Repeat steps 1-4 until a terminal state is reached.

Optimal decisions under chance

- Minimax → Expectiminimax algorithm

EXPECTIMINIMAX(s) =

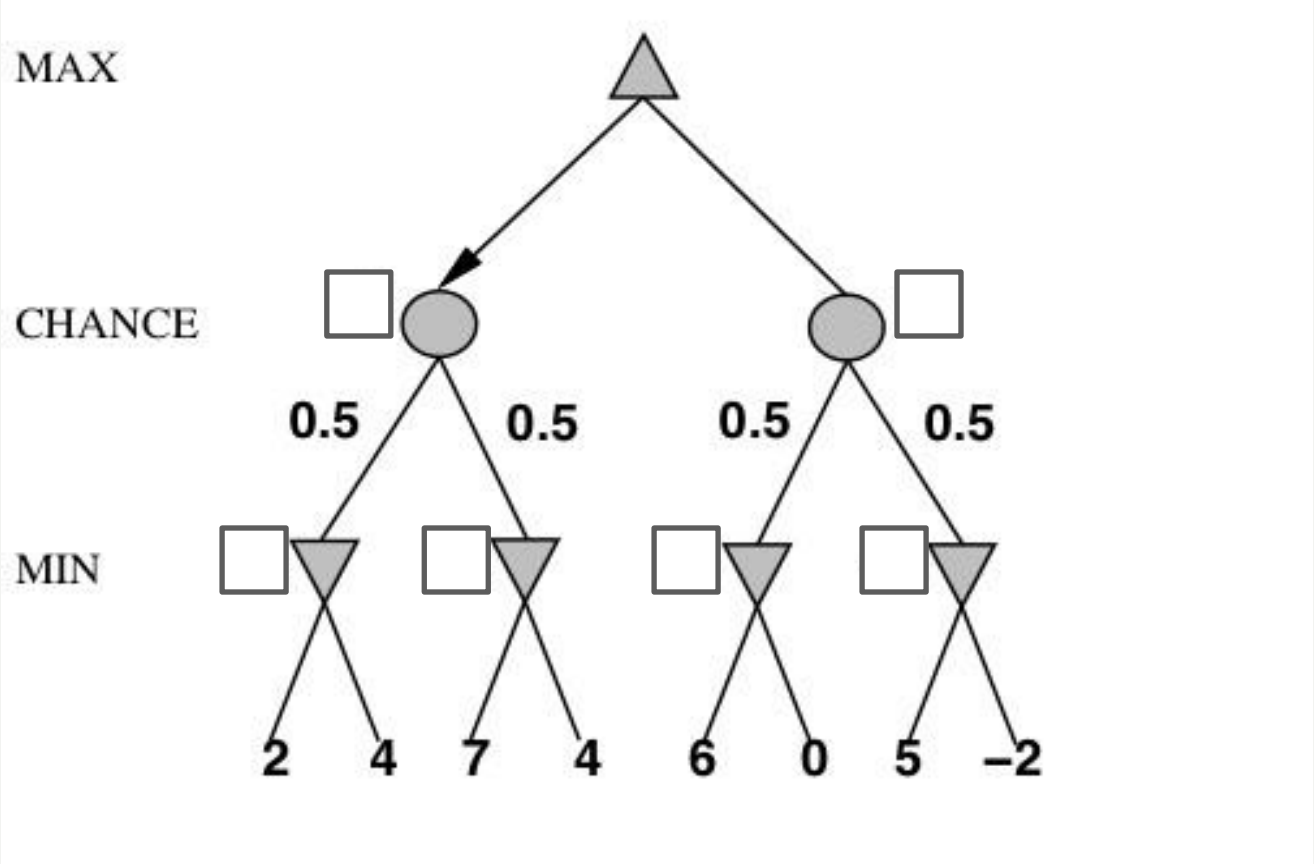
UTILITY$(s,$ MAX$)$	if IS-TERMINAL(s)
max $_a$ EXPECTIMINIMAX(RESULT(s,a))	if TO-MOVE(s) = MAX
min $_a$ EXPECTIMINIMAX(RESULT(s,a))	if TO-MOVE(s) = MIN
$\sum_r P(r)$ EXPECTIMINIMAX(RESULT(s,r))	if TO-MOVE(s) = CHANCE

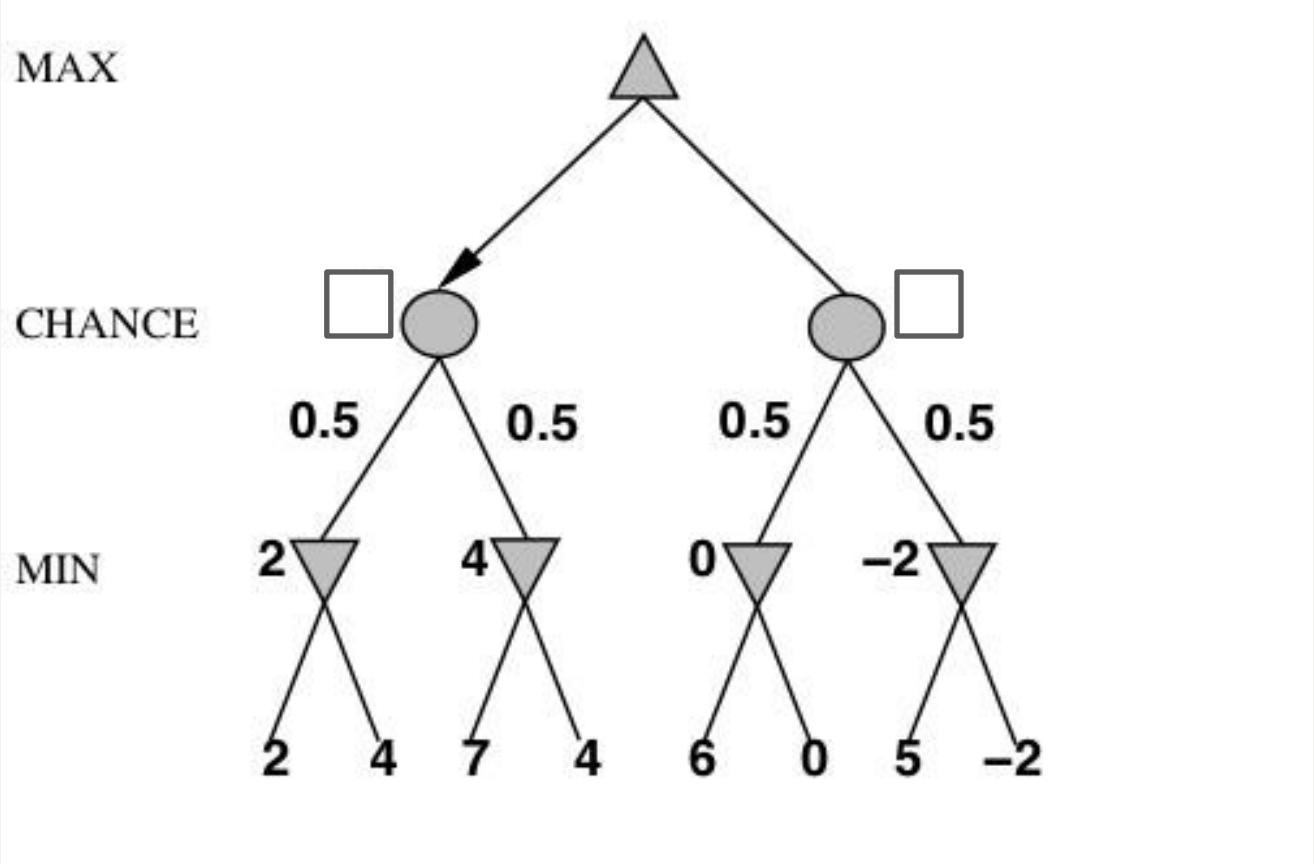
expected value

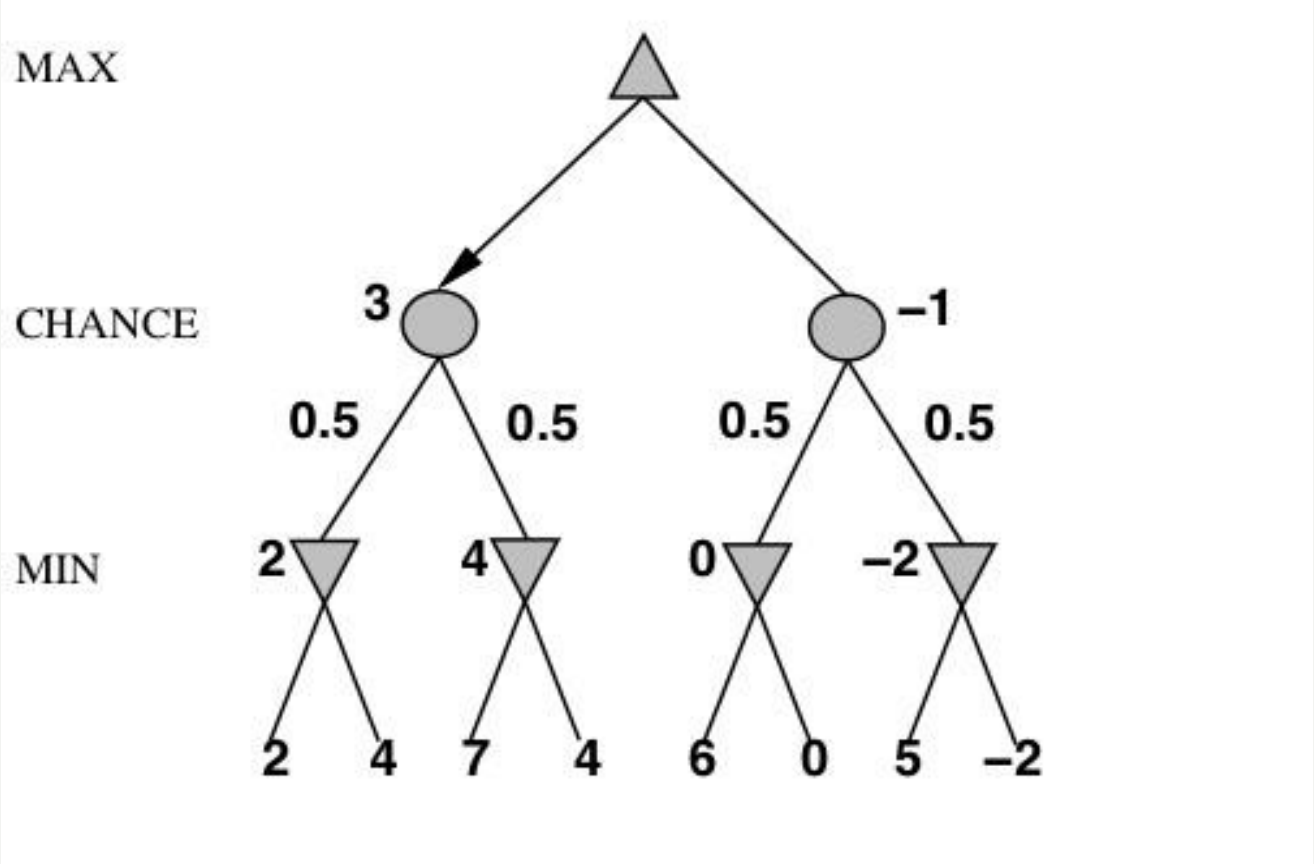
one possible dice roll

Examples

- Coin-flip game
- Rules:
  1. MAX makes one of the two moves
  2. MIN flips a coin
  3. MIN makes one of the two moves

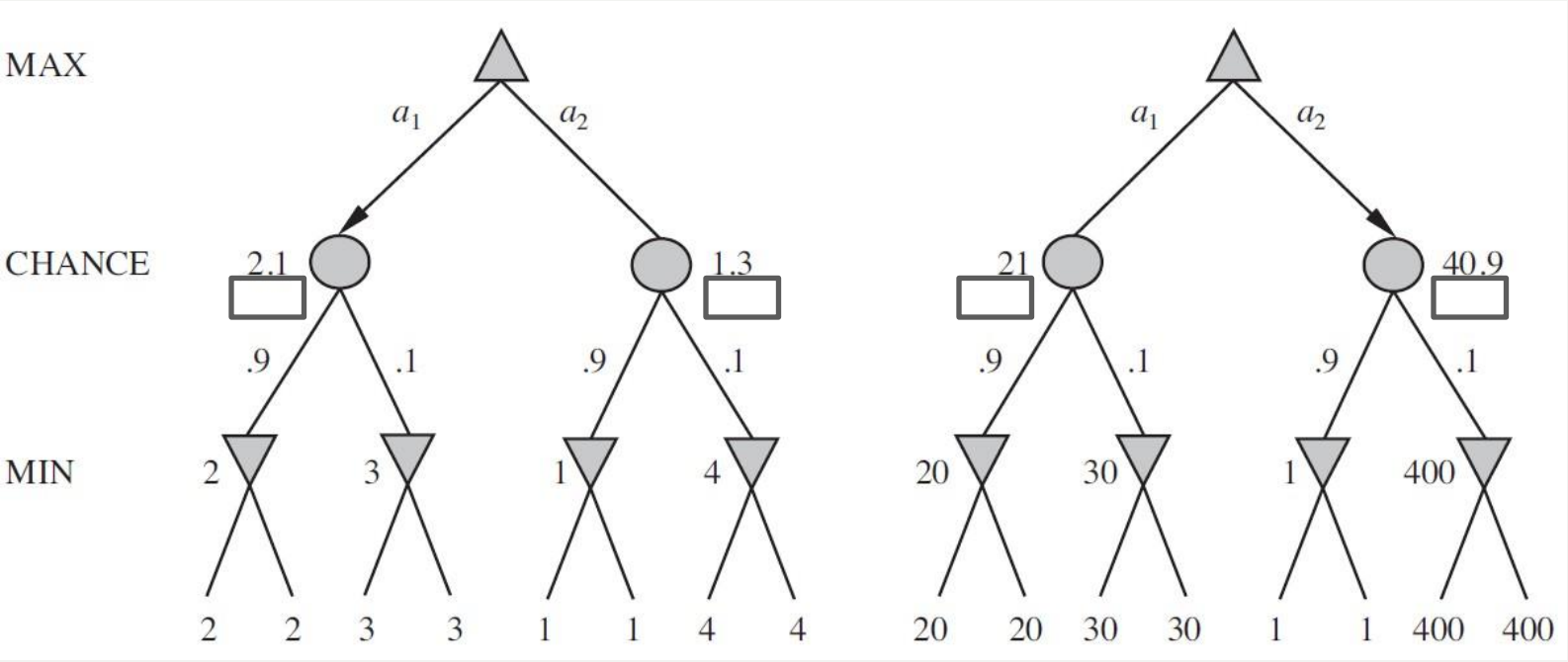

MAX
CHANCE
0.5
0.5
0.5
0.5
MIN
2
4
7
4
6
0
5
–2

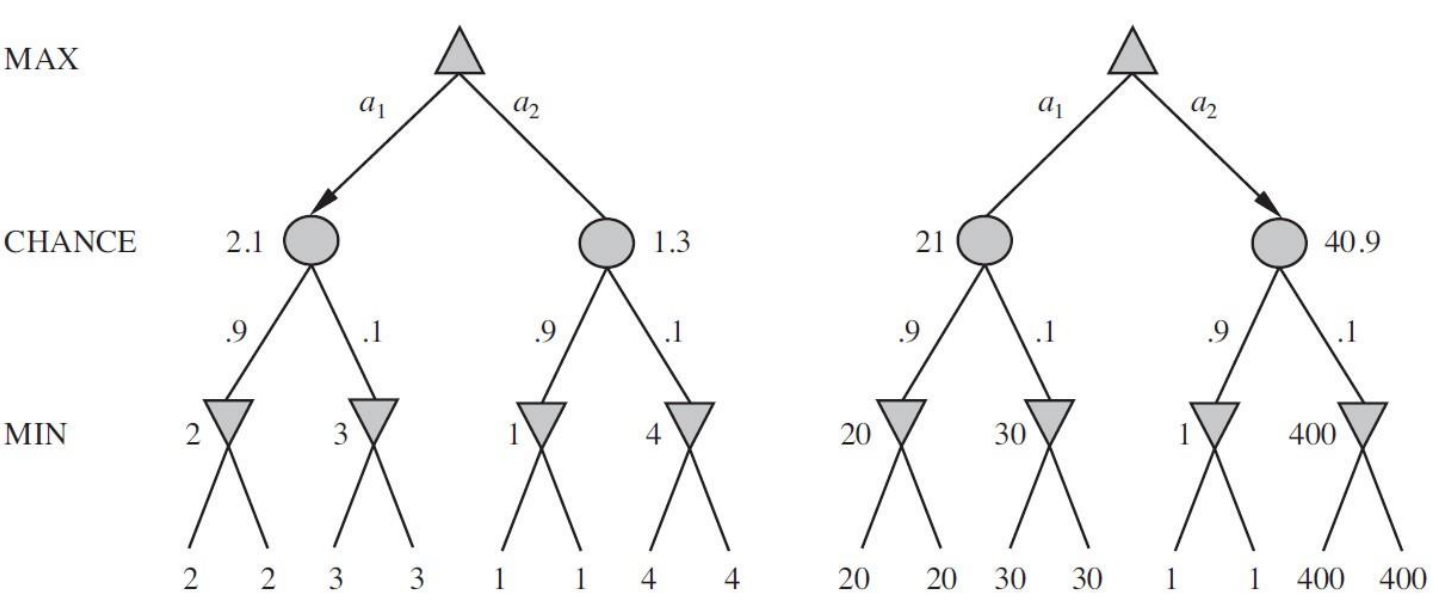

MAX
CHANCE
MIN
0.5
0.5
0.5
0.5
2
4
0
−2
2
4
7
4
6
0
5
−2


MAX
CHANCE
MIN
3
−1
0.5
0.5
0.5
0.5
2
4
0
−2
2
4
7
4
6
0
5
−2

With an evaluation function that assigns the values [1, 2, 3, 4] to the leaves, move a1 is best; with values [1, 20, 30, 400], move a2 is best. Hence, the program behaves totally differently if we make a change in the scale of some evaluation values! It turns out that to avoid this sensitivity, the evaluation function must be a positive linear transformation of the probability of winning from a position (or, more generally, of the expected utility of the position).

- Optimal decisions under expectiminimax is different from minimax
  - Scaling of utility values while other numbers are kept fixed





- Without chance node, optimal minimax decision would've been the same before and after scaling
- Chance nodes introduces additional branching factor
  - If n = number of possible outcomes, then $O((bn)^m)$
  - Backgammon: $((20 \sim 4000) \times 21)^m$
  - 21 = number of possible outcomes for (die 1, die 2)